1. 함수는 입출력

함수가 뭐야?



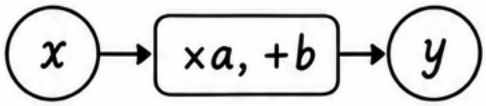
함수는 무언가를 입력받아, 그에 따른 결과를 출력하는 규칙이나 장치라고 생각하면 된다. 대표적인 예가 자판기다. 자판기에 돈을 넣고 음료를 선택하면 음료수가 나온다. 떡을 만드는 기계도 비슷하다. 쌀을 넣으면 떡이 되어 나온다. 조금 더 그럴듯하게 표현하면, 함수는 입력을 받아 출력을 만들어 내는 것이라고 할 수 있다. 처음에는 함수가 '입력과 출력의 관계'라는 것만 이해해도 충분히 유용하다. 컴퓨터도 키보드나 마우스의 입력을 받아 그에 맞는 동작을 한다. 스마트폰도 마찬가지다. 이런 관점에서 보면 주변의 많은 기계를 함수에 비유할 수 있다. 그러고 보면 사람도 함수처럼 생각해 볼 수 있다. 우리는 음식을 먹고 소화한 뒤 배설한다. 먹는 것이 입력이고, 소화와 배설의 과정이 출력으로 이어지는 셈이다. 말하는 것도 함수와 비슷하다. 머릿속에서 무언가를 생각하고, 그것을 목소리로 표현한다. 생각이 입력이라면 말소리는 출력이다. 듣는 것도 마찬가지다. 소리를 입력받아 그 말의 의미를 이해한다. 따라서 대화는 한 사람의 '말하는 함수'와 다른 사람의 '듣는 함수'가 연결된 과정이라고 볼 수 있다. 더 작은 단위로 들어가면 신경세포도 함수처럼

생각할 수 있다. 다른 신경세포에서 신호를 받아 처리한 뒤, 또 다른 신경세포로 신호를 보낸다. 물론 이런 예들은 함수를 쉽게 이해하기 위한 비유다. 수학에서 함수는 좀 더 정확하게, 각 입력에 하나의 출력이 정해지는 규칙을 뜻한다.

함수는 입력과 출력 사이의 규칙

1. 일차함수

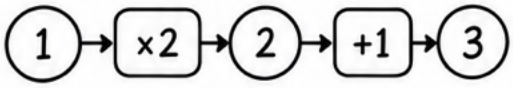
$$y = ax + b$$



입력 규칙 출력

2. 값을 넣어 보면

$$y = 2x + 1$$



$x = 1 \rightarrow y = 3$

3. 함수의 이름과 입력

$$f(x)$$

함수 이름 입력

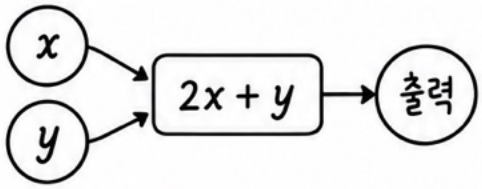
이름은 g, h도 가능

4. 같은 관계, 다른 표기

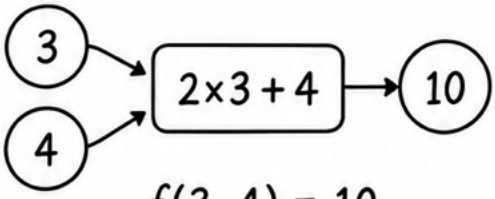
$$y = ax + b$$
$$\updownarrow$$
$$f(x) = ax + b$$
$$\updownarrow$$
$$y = f(x)$$

5. 입력이 두 개라면

$$f(x, y) = 2x + y$$

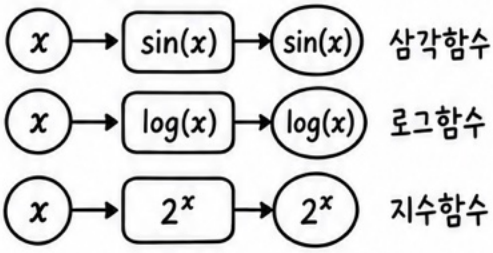


6. 두 값을 넣어 보면



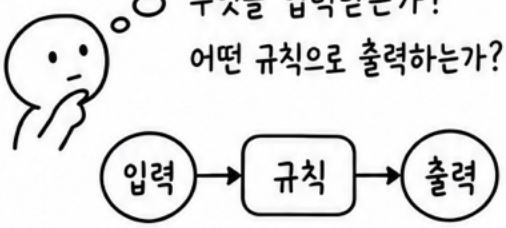
$f(3, 4) = 10$

7. 규칙은 다양하다



로그함수는 $x > 0$

8. 함수를 이해하는 두 질문



우리가 함수를 가장 먼저 배우는 곳은 아마 수학일 것이다. 대표적인 예가 일차함수다.

$y = ax + b$

이 식에서 입력은 x, 출력은 y 다. 보통 입력을 x, 출력을 y 라고 부르지만 반드시 그래야 하는 것은 아니다. 예를 들어 다음과 같은 함수가 있다고 하자.

$y = 2x + 1$

여기에 $x = 1$ 을 입력하면, 입력한 1 에 2 를 곱하고 다시 1 을 더한다. 따라서 출력은 $y = 3$ 이다. 이 함수는 입력값을 받아 2 를 곱하고 1 을 더한 뒤, 그 결과를 출력한다. $y = ax + b$ 는 다음과 같이 쓰기도 한다.

$f(x) = ax + b$

앞부분이 조금 달라졌다. 먼저 f(x)부터 읽어 보자. f 는 함수의 이름이다. 함수의 이름은 꼭 f 일 필요는 없다. g 나 h 를 사용해도 된다. 함수 이름 뒤에 붙은 괄호는 괄호 안의 값이 그 함수의 입력이라는 뜻이다. 따라서 f(x)는 다음과 같이 이해하면 된다. 함수의 이름은 f 이고, 입력은 x 다. 이제 오른쪽의 $ax + b$ 를 보자. 이것은 함수가 입력값을 가지고 실제로 하는 일을 나타낸다. 동시에 그 계산으로 만들어지는 출력값이기도 하다. 따라서 $f(x) = ax + b$ 라는 식 전체는 다음과 같은 의미다. 이름이 f 인 함수가 있다. 이 함수는 x 를 입력받아 $ax + b$ 를 출력한다. $y = ax + b$ 와 $f(x) = ax + b$ 는 결국 같은 내용을 담고 있는데, 이를 $y = f(x)$라고 쓰면 그 관계가 좀 더 분명해진다. $y = ax + b$ 는 x 와 y 의 관계를 바로 나타내고, $f(x) = ax + b$ 는 그 함수의 이름이 f 라는 사실까지 함께 보여 준다.

지금까지 살펴본 함수는 입력값이 하나였다. 하지만 함수에는 입력이 두 개 이상 들어갈 수도 있다. 입력이 두 개라면 다음과 같이 쓸 수 있다.

$f(x, y)$

물론 입력의 이름을 반드시 x 와 y 라고 할 필요는 없다. p 와 q 라고 불러도 전혀 문제가 없다. 그렇다면 오른쪽의 출력 부분에는 무엇을 적어야 할까? 입력값들을 이용해 어떤 결과를 만드는 규칙을 적으면 된다. 예를 들어 다음 함수를 보자.

$f(x, y) = 2x + y$

이 함수는 x 와 y, 두 개의 값을 입력받는다. 그리고 첫 번째 입력값 x 에 2 를 곱한 뒤, 두 번째 입력값 y 를 더해 그 결과를 출력한다. 예를 들어 x = 3, y = 4 를 입력하면 다음과 같다.

f(3, 4) = 2×3 + 4 = 10

따라서 출력은 10 이다. 결국 함수를 이해하려면 두 가지를 알면 된다. 첫째, 무엇을 입력받는가? 둘째, 그 입력을 이용해 어떤 출력을 만드는가? 그렇다면 삼각함수, 로그함수, 지수함수는 무엇일까? 이 함수들도 기본적인 원리는 똑같다. 사인함수는 다음과 같이 쓸 수 있다.

f(x) = sin(x)

로그함수는 다음과 같다.

f(x) = log(x)

밑이 2 인 지수함수는 다음과 같이 쓸 수 있다.

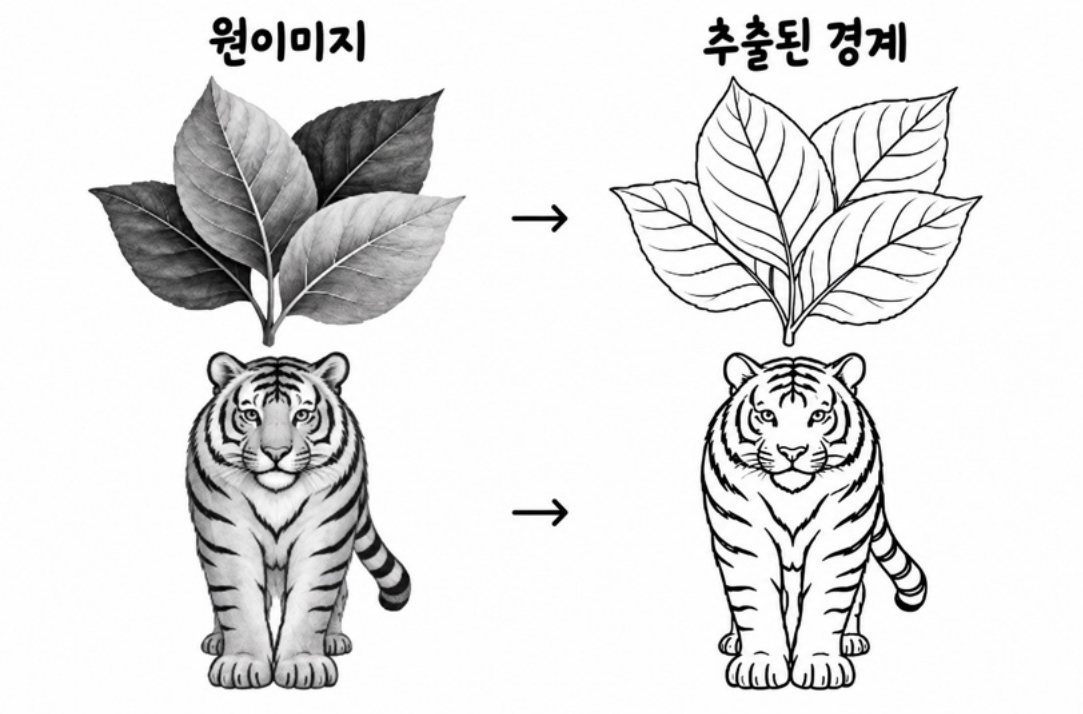
f(x) = 2^x

이름이 무엇이든 기본적으로는 모두 같다. 입력값을 넣으면, 각 함수가 정해진 방식으로 그 값을 처리해 결과를 출력한다. 따라서 삼각함수나 로그함수, 지수함수라는 이름이 어렵게 느껴지더라도 처음에는 이렇게 생각하면 된다. 값을 하나 넣었더니, 함수가 그 값을 자기만의 규칙으로 이리저리 처리해서 결과를 내놓는구나. 함수는 결국 입력과 출력 사이의 규칙이다.

2. 미분은 차이와 변화

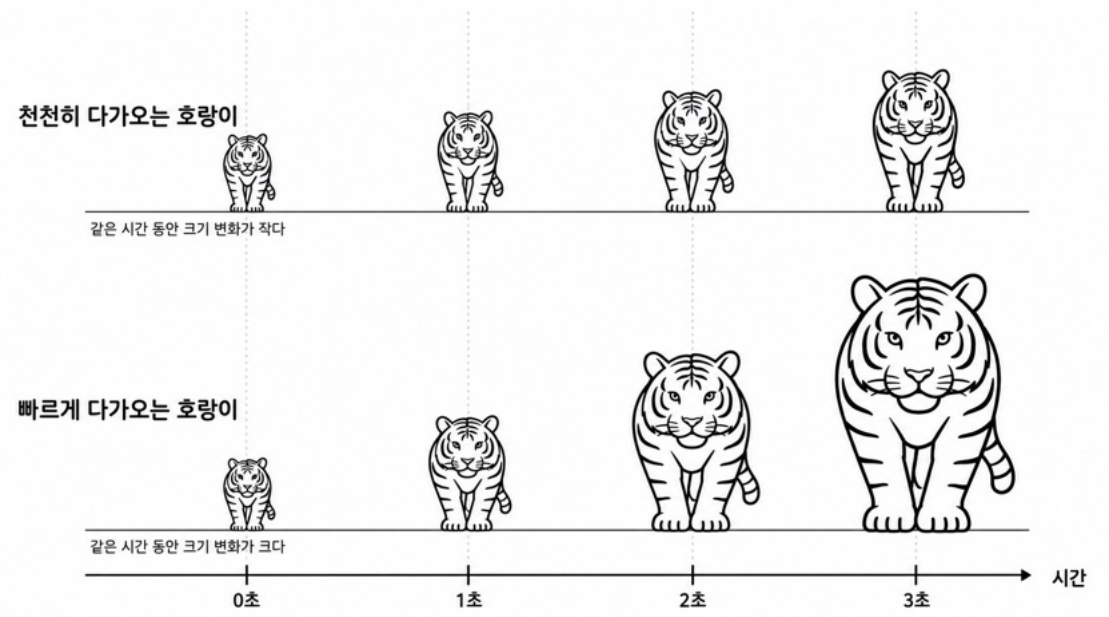
눈을 뜬 순간, 망막에 있는 수백만 개의 시각세포로 각기 다른 빛 신호들이 쏟아진다. 이 원시 데이터를 그대로 뇌로 넘긴다면, 아무리 큰 뇌라도 감당할 수 없을 것이다.

하지만, 이 세포는 한 지점의 밝기 그 자체가 아니라, 그 지점과 주변 지점 사이의 밝기 차이에 반응하도록 설계되어 있다. 그래서 비교적 균일한 배경에는 거의 반응하지 않고, 밝기가 급격히 바뀌는 경계선에서만 강하게 반응한다.



결과적으로 뇌에 전달되는 것은 장면 전체의 픽셀 값이 아니라, 사물의 윤곽선이라는 압축된 스케치다. 맹수의 얼룩무늬든, 나뭇가지 사이의 형체든, 우리는 채워진 면이 아니라 경계의 집합으로 그것을 인식한다. 이 경계의 패턴만으로도 뇌는 저장된 형태 기억과 비교해 그것이 맹수인지 아닌지를 순식간에 판별해낼 수 있다. 정보량은 원본의 극히 일부로 줄었지만, 판단에 필요한 핵심은 고스란히 남아 있는 것이다.

쉽게 말하면 어디에서 차이가 많이 나는지를 이용하는 것이다. 이것이 미분이다. 조금더 자세히 말하면 공간 미분이다. 공간 미분이란 공간을 따라 얼마나 급하게 변하는지이다. 그래서 어떤 이미지에서 공간미분이 큰 부분을 윤곽이라고 한다. 공간을 따라 얼마나 급변하는지 보는게 공간 미분이라면, 시간 미분도 있다. 시간 미분은 이 원리를 시간축으로 확장한 것이다. 일상의 용어로 우리는 이걸 변화라 하기도 하고 속도라고 하기도 한다.



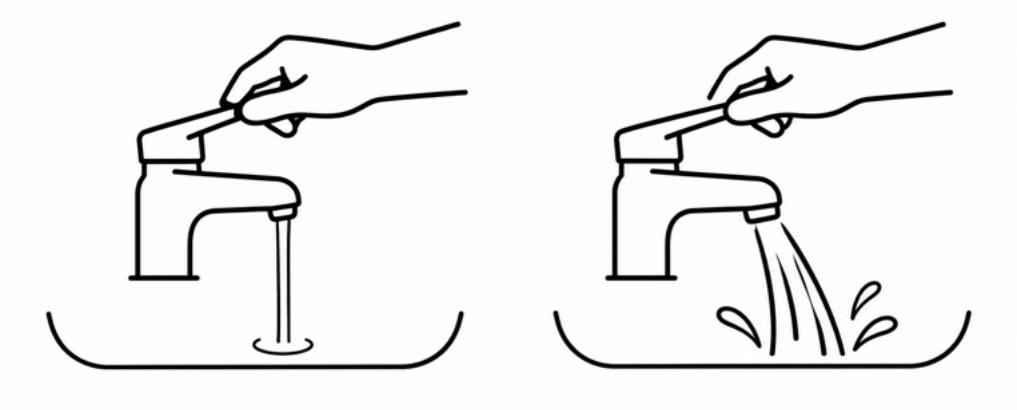
정지한 사물은 배경 속에 묻혀 거의 신호를 만들지 않지만, 움직이는 사물은 즉시 신경 반응의 급증을 일으킨다. 이것이 우리가 정지한 배경보다 움직이는 물체를 압도적으로 빨리 알아채는 이유다. 더 나아가 멀리서 다가오는 맹수처럼 크기가 점점 커지는 대상에 대해서는, 망막 위 상의 팽창 속도 자체를 계산하는 신경 회로가 존재한다. 이 팽창률은 대상이 나에게 도달하기까지 남은 시간과 수학적으로 직결되어 있어서, 뇌는 상대의 실제 거리나 속도를 몰라도 "충돌까지 얼마 남았는가"를 즉각적으로, 그리고 매우 정확하게 추정할 수 있다.

정리하면 뇌는 세상을 있는 그대로 기록하는 저장장치가 아니라, 변화가 일어나는 지점만을 골라내는 미분기다. 공간 미분은 "무엇인가"를 압축된 윤곽으로 알려주고, 시간 미분은 "그것이 어떻게 움직이는가"를 압축된 변화율로 알려준다. 정지해 있고 균일한 것은 정보가 아니다. 정보는 언제나 차이에서, 그리고 변화에서 발생한다. 눈앞에 맹수가 나타났을 때 우리에게 필요한 것은 그 장면의 완전한 사본이 아니라, 그것이 무엇이며 얼마나 빨리 다가오는가라는 단 두 개의 압축된 답이었다. 수백만 년의 생존압은 뇌로 하여금 바로 그 두 가지 질문에 최적화된 방식으로 세상을 읽어내도록 만들었다. 그것이 공간 미분과 시간 미분이라는, 지극히 경제적이면서도 정교한 진화의 해법이다.

미분은 애초에 '입력이 바뀌면 출력이 얼마나 바뀌는가'를 다루는 개념이다. 즉 1 장에서 살펴본 함수를 전제로 한다. 앞서 살펴본 공간 미분과 시간 미분도 마찬가지다. 공간 미분은

망막 위의 위치를 입력, 그 위치와 주변 사이의 밝기 차이를 출력으로 하는 함수의 미분이고, 시간 미분은 시각을 입력, 그 시각의 밝기(혹은 상의 크기)가 이전 순간과 비교해 얼마나 달라졌는가를 출력으로 하는 함수의 미분이다. 이처럼 어떤 관계든 입력과 출력으로 나눌 수만 있다면, 그 사이에는 미분을 물을 수 있다. 어떤 입력이 출력을 얼마나 크게 바꾸어놓는가를 묻는 순간, 우리는 이미 미분적으로 사고하고 있는 셈이다.

미분의 개념은 반드시 시간이나 공간에만 적용되는 것이 아니다. 일상에서는 이를 흔히 영향력, 민감도, 때로는 책임이라는 말로 표현한다. 어떤 수도꼭지는 손잡이를 조금만 돌려도 물이 갑자기 많이 나온다. 조절하기가 쉽지 않다. 여기서 손잡이를 돌린 정도를 입력, 흘러나오는 물의 양을 출력이라고 하는 함수를 생각해보자. 입력을 조금 바꾸었는데 출력이 크게 변했으므로, 손잡이를 돌린 정도에 대한 물의 양의 미분값이 크다고 할 수 있다. 물의 양이 손잡이의 움직임에 매우 민감하다고 표현해도 같은 뜻이다.



우유를 한 모금만 마셔도 배탈이 심하게 나는 사람이 있다. 이때 우유의 양은 입력이고, 몸의 반응은 출력이다. 아주 적은 양에도 몸이 크게 반응한다면, 우유 섭취량에 대한 몸의 반응의 미분값이 크다. 원인의 관점에서 보면 우유가 몸에 미친 영향력이 크다고 할 수 있고, 결과의 관점에서 보면 몸이 우유에 민감하다고 할 수 있다. 같은 현상을 어느 쪽에서 바라보느냐에 따라 영향력과 민감도로 달리 부를 뿐이다.



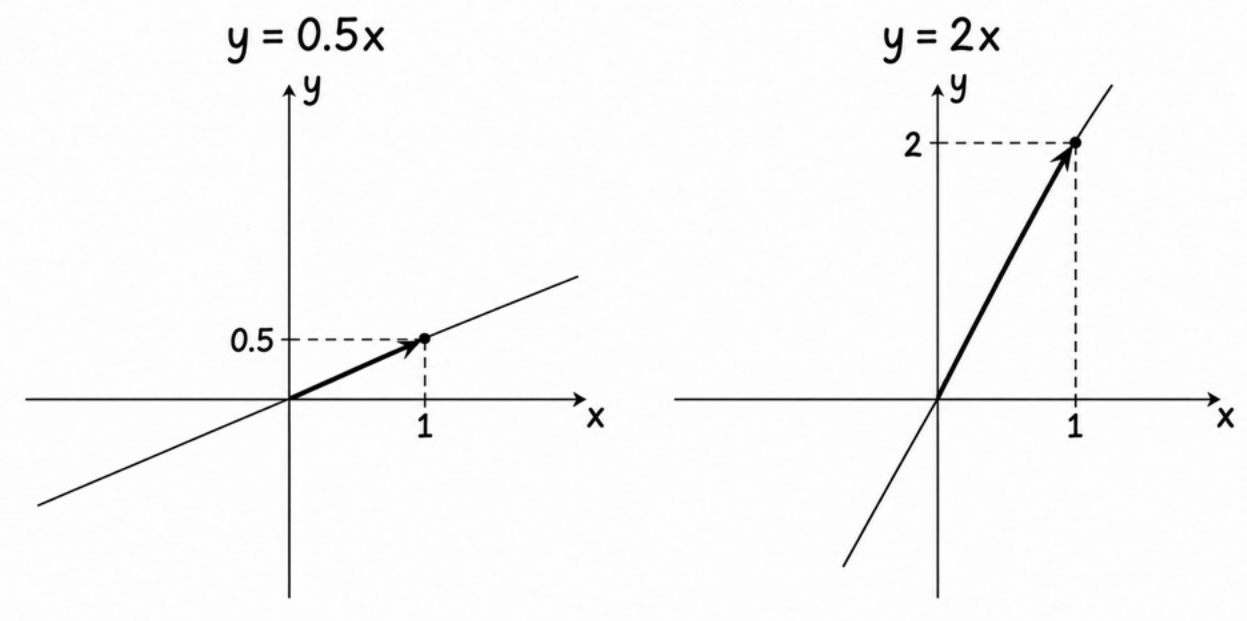
다른 사람이 무심코 한 말에도 쉽게 상처받는 친구가 있다고 해 보자. 다른 사람의 말은 입력이고, 친구의 마음에 일어나는 변화는 출력이다. 작은 말 한마디에도 마음이 크게 흔들린다면, 다른 사람의 말에 대한 그 친구 마음의 변화율이 크다고 할 수 있다. 그 친구의 마음은 말에 민감하게 반응하는 것이다.



이처럼 미분은 입력의 작은 변화가 출력에 얼마나 큰 변화를 일으키는지를 나타낸다. 입력의 관점에서 보면 출력에 미치는 '영향력'이고, 출력의 관점에서 보면 입력에 반응하는 '민감도'다. 입력이 결과에 얼마나 관여했는지를 강조하면 비유적으로 '책임'이라고 표현할 수도 있다. 결국 영향력, 민감도, 책임은 서로 다른 세 가지 개념이 아니라, 하나의 미분값을 원인과 결과 중 어느 쪽에서 바라보느냐에 따라 붙는 이름일 뿐이다.

그런데 우리는 미분을 ‘x 변화량을 y 변화량으로 나눈 것’이라고 배우기도 했다. ‘기울기’라는 말도 들어 보았을 것이다. 모두 같은 개념을 서로 다른 방식으로 설명한 말이다.

두 그래프를 비교해 보자. 기울기는 입력 x 가 변할 때 출력 y 가 얼마나 변하는지를 나타낸다. 변화량으로 쓰면 다음과 같다. Δy/Δx



왼쪽 그래프에서는 입력 x 가 1 만큼 증가할 때 출력 y 가 0.5 만큼 증가한다. 따라서 기울기는 0.5 다. 오른쪽 그래프에서는 입력 x 가 1 만큼 증가할 때 출력 y 가 2 만큼 증가한다. 따라서 기울기는 2 다. 같은 크기의 입력 변화가 주어졌지만, 왼쪽의 출력은 0.5 만큼 변했고 오른쪽의 출력은 2 만큼 변했다.

따라서 입력이 출력에 미치는 영향력은 오른쪽이 더 크다고 할 수 있다. 출력의 관점에서 보면, 왼쪽은 입력의 변화에 덜 민감하고 오른쪽은 더 민감하다. 같은 입력에도 오른쪽의 출력이 훨씬 크게 반응하기 때문이다. 그래서 기울기, 변화율, 영향력, 민감도는 모두 미분의 같은 성질을 서로 다른 관점에서 표현한 말이다. '책임' 역시 원인이 결과에 얼마나 크게 관여했는지를 설명하기 위한 비유로 이해할 수 있다. (주의: 엄밀히 말하면 Δy/Δx 는 일정한 구간에서의 평균변화율이다. 미분은 Δx 를 한없이 작게 줄였을 때의 순간변화율을 뜻한다. 직선에서는 기울기가 어디서나 일정하므로 평균변화율과 미분값이 같다.)

3. 행복은 미분

행복은 미분이다

재산의 크기보다 변화와 비교 기준에 주목하는 비유

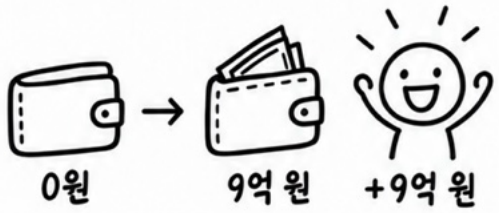
갑작스러운 증가



익숙해진 풍요



출발점과 비교



정점과 비교



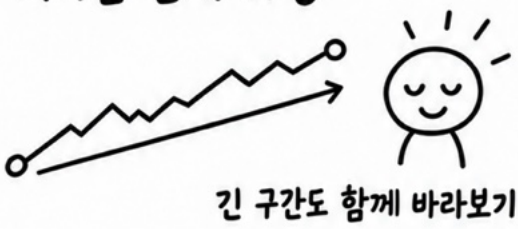
갚아 온 길을 보면



남은 빚만 보면



지나온 전체 과정



최근의 변화



설명을 위한 비유입니다. 기준점과의 차이는 변화량이며, 변화율은 이를 시간 간격으로 나눈 값입니다.

우리는 늘 행복을 원한다. 그런데 행복이라는 것을 우리는 은연중에 절대적인 값처럼 여겨왔는지도 모른다. 재산이 많으면 행복하고 적으면 불행하다고, 마치 재산이라는 입력값 하나가 행복이라는 출력값을 그대로 결정하는 것처럼 생각한다. 하지만 정말 그럴까? 만약 행복이 절대적인 값이라면, 1000 억 원을 가진 사람은 언제나 행복해야 마땅하다. 재산의 크기가 곧 행복의 크기라면, 큰 부자는 늘 큰 행복 속에서 살아야 한다. 그러나 우리는 안다. 엄청난 부를 가진 사람도 얼마든지 불행할 수 있고, 반대로 가진 것이 많지 않은 사람도 얼마든지 행복할 수 있다는 것을. 이 모순은 행복이 절대적인 값이 아니라 상대적인 값이라는 사실을, 즉 재산이라는 함수의 절대적인 출력값이 아니라는 사실을 보여준다.

앞서 우리는 미분이 '입력이 바뀔 때 출력이 얼마나 바뀌는가'를 다루는 개념이라는 것을 살펴보았다. 망막의 신경절세포도, 수도꼭지의 물줄기도, 우유 한 모금에 예민한 몸도, 말 한마디에 흔들리는 마음도 모두 이 하나의 틀 안에서 설명되었다. 행복도 이 틀을 벗어나지 않는다. 이번에는 시간을 입력, 그 시점의 재산을 출력으로 하는 함수를 생각해보자. 재산(t)이라는 이 함수는 시간이 흐르면서 재산이 어떻게 늘고 줄어왔는지를 보여준다. 이는 앞서 살펴본 시간 미분, 즉 시각을 입력으로 하고 그 시각의 밝기나 상의 크기를 출력으로 삼았던 함수와 정확히 같은 구조다. 그런데 우리가 실제로 느끼는 행복이란 이 함수 자체의 값, 즉 지금 재산이 얼마인가가 아니라 이 함수의 미분값, 다시 말해 시간에 따라 재산이 얼마나 빠르게 늘거나 줄고 있는가에 가깝다. 심리학에서는 이와 비슷한 현상을 '적응 수준'이나 '쾌락의 쳇바퀴'라는 말로 설명하기도 하지만, 결국 하려는 말은 같다. 행복은 재산의 절대량이 아니라 시간에 대한 재산의 변화율에서 나온다는 것, 다시 말해 행복은 미분이라는 것이다.

예를 들어보자. 재산이 하나도 없던 내가 어느 날 갑자기 로또에 당첨되어 10 억 원을 손에 쥐었다고 하자. 통장에 10 억이라는 숫자가 찍히는 순간 뛸 듯이 기쁘다. 그런데 이 기쁨은 정확히 어디에서 오는 것일까. 흔히는 '10 억을 가졌기 때문에' 행복하다고 생각한다. 하지만 조금 더 정확히 말하면, 이 행복은 지금 10 억을 가지고 있다는 사실 자체가 아니라 어제까지의 재산과 비교했을 때 그만큼의 변화가 있었기 때문에 생긴다. 어제의 재산인 0 원을 기준점으로 놓고 오늘의 10 억 원을 그 기준점과 비교하는 순간, 그 차이, 즉 미분값이 고스란히 행복의 크기로 나타나는 것이다. 만약 애초에 10 억을 가지고 태어나 평생 그 상태를 유지해온 사람이 있다면, 이 사람에게는 통장의 10 억이라는 숫자가 아무런 감흥을 주지 못한다. 변화가 없으므로 미분값이 0 이고, 미분값이 0 이라는 것은 그 순간 새로

발생하는 행복도 불행도 없다는 뜻이다. 결국 행복을 만든 것은 10 억이라는 절대적인 크기가 아니라 0 에서 10 억으로 건너간 변화의 폭이었던 셈이다. 문제는 이 기준점이 시간이 지나면 슬그머니 옮겨간다는 데 있다. 로또에 당첨된 지 1 년이 지나면 마음은 더 이상 '재산이 없던 어제'를 기준으로 삼지 않는다. 이제는 '10 억을 가지고 있던 어제'가 새로운 기준점이 된다. 돈을 아무리 많이 벌었어도 더 벌려고 하는 이유도 여기에 있다.

이 상태에서 투자에 실패하거나 여기저기 돈을 쓰다가 1 억 원이 사라져 9 억 원만 남았다고 해보자. 절대적인 재산의 크기로 보면 이 사람은 여전히 처음보다 9 억 원이나 많은 돈을 가지고 있다. 그런데도 이 사람은 불행을 느낀다. 이번에 비교의 기준점이 된 것은 '10 억을 가졌던 어제'이고, 그 기준점과 오늘의 9 억 원 사이에는 1 억 원만큼의 감소, 즉 재산이라는 출력값의 음수 변화가 있기 때문이다. 시간이 흐르는 동안 재산이 줄어들었으므로 이 구간에서의 미분값, 즉 변화율은 음수다. 미분값이 음수라는 것은 감정이 음의 방향으로 반응한다는 뜻이고, 우리는 이것을 불행이라고 부른다. 재산은 압도적으로 많이 남아 있는데도, 그 절대량이 아니라 최근의 변화율, 즉 미분값이 감정을 결정하기 때문에 벌어지는 일이다.

그런데 이 미분값을 계산하는 기준점을 조금만 바꾸어도 완전히 다른 그림이 나온다. 기준점을 '10 억을 가졌던 어제'가 아니라 애초에 로또에 당첨되기 전, 재산이 하나도 없던 시절로 되돌리면 어떻게 될까. 그 시절과 지금의 9 억 원을 비교하면, 이 사람은 여전히 9 억 원어치의 순수한 증가를 이루어낸 셈이다. 똑같은 9 억 원을 두고도 기준점을 최근의 정점으로 잡느냐 출발점으로 잡느냐에 따라 미분값의 부호가 완전히 뒤바뀐다. 하나는 음수라서 불행이고 다른 하나는 양수라서 행복이다. 흔히 '초심을 잃지 말라'는 말을 한다. 이 말을 미분의 언어로 다시 풀면, 감정의 미분을 계산할 때 쓰는 기준점을 욕심이 가장 부풀었던 정점이 아니라 처음 출발했던 지점으로 삼으라는 뜻이 된다. 기준점을 어디에 두느냐에 따라 똑같은 재산이 불행의 근거도 되고 행복의 근거도 될 수 있다는 것을, '초심'이라는 말은 이미 알고 있었던 셈이다.

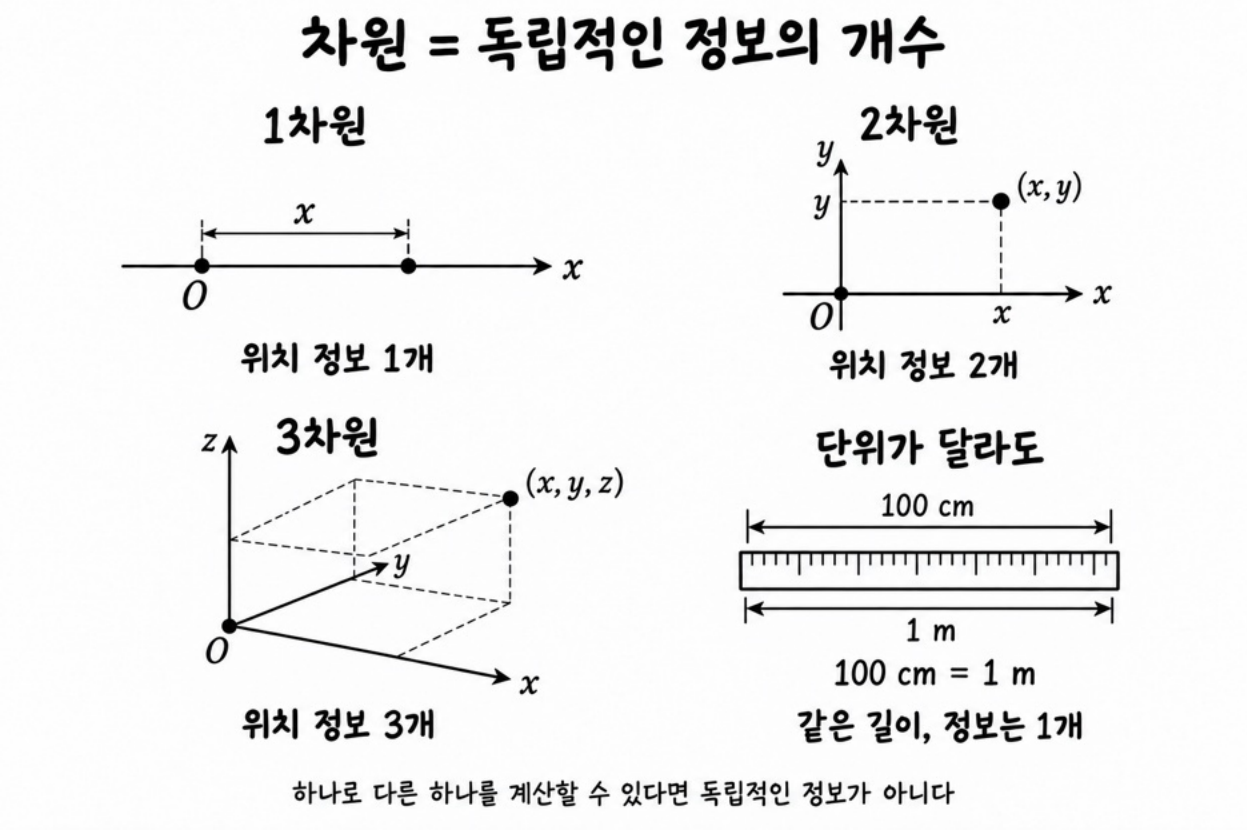
이 원리는 재산이 늘어나거나 줄어드는 경우만이 아니라, 빚처럼 마이너스에서 시작하는 경우에도 그대로 적용된다. 어떤 사람에게 1 억 원의 빚이 있었다고 하자. 이 사람이 한 해 동안 열심히 일해서 5 천만 원을 갚았다. 결과만 놓고 보면 이 사람에게는 여전히 5 천만 원의 빚이 남아 있다. 이 사람이 감정의 미분을 계산할 때 기준점을 빚이 전혀 없는 상태, 즉 0 원으로 잡는다면 어떻게 될까. 그 기준점과 지금을 비교하면 여전히 5 천만 원어치의

마이너스이므로, 이 사람은 열심히 빚을 갚고도 불행하다고 느낀다. 반면 기준점을 1 년 전, 그러니까 1 억 원의 빚이 있었던 시점으로 잡으면 전혀 다른 계산이 나온다. 1 억 원의 빚에서 5 천만 원의 빚으로, 부채가 절반으로 줄어드는 양의 변화, 즉 플러스의 미분값을 얻은 것이다. 이 사람은 같은 5 천만 원의 빚을 두고도 얼마든지 행복해질 수 있다. 기준점을 어디로 잡느냐에 달린 문제이기 때문이다.

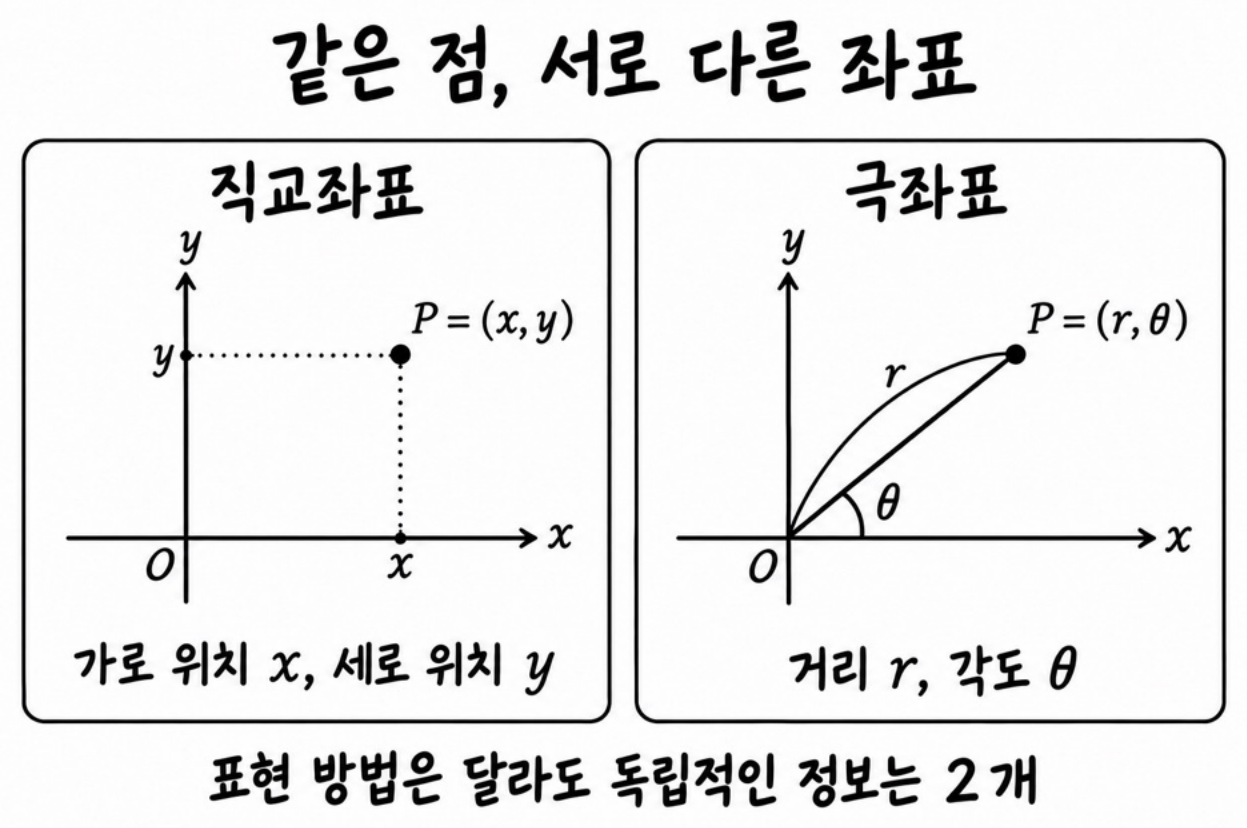
이처럼 우리가 삶에서 실제로 계산하는 감정의 미분값은 언제나 어떤 기준 시점 위에서 이루어진다. 그런데 그 기준 시점을 정하는 일은 수학처럼 명확한 규칙을 따르지 않는다. 그것은 상당 부분 우리가 무엇을 기억하고 무엇을 기억에서 지우는가에 달려 있다. 정점의 순간만을 또렷하게 기억하는 사람은 그 정점을 기준점으로 삼기 쉽고, 그 아래로 조금만 내려와도 쉽게 불행해진다. 반면 자신이 걸어온 길의 출발점을 잊지 않는 사람은 작은 하락에도 쉽게 흔들리지 않는다. 결국 기준점을 어디에 두느냐는 단순한 계산의 문제가 아니라 무엇을 기억하려 하는가라는 태도의 문제이며, 나아가 삶을 대하는 자세의 문제이기도 하다.

이는 앞서 살펴본 평균변화율과 순간변화율의 관계와도 닮아 있다. 아주 최근의 변화, 이를테면 어제와 오늘의 차이만을 좁게 비교하면 순간적인 감정의 기복에 휘둘리기 쉽다. 반면 출발점부터 지금까지의 긴 구간을 기준으로 잡으면, 그 사이의 작은 오르내림은 상대적으로 완만해진다. 짧은 구간에서의 순간변화율에 일희일비하지 않고, 인생 전체라는 긴 구간에서의 평균변화율을 함께 바라보는 것. 그것이 어쩌면 쉽게 흔들리지 않는 행복에 조금 더 가까이 다가가는 방법일지도 모른다. 같은 크기의 재산, 같은 크기의 빚을 두고도 누군가는 불행하고 누군가는 행복하다면, 그 차이는 재산의 크기가 아니라 그 사람이 마음속에 어떤 기준점을, 즉 감정의 미분이 시작되는 원점을 세워 두었는가에 달려 있는 셈이다.

4. 차원

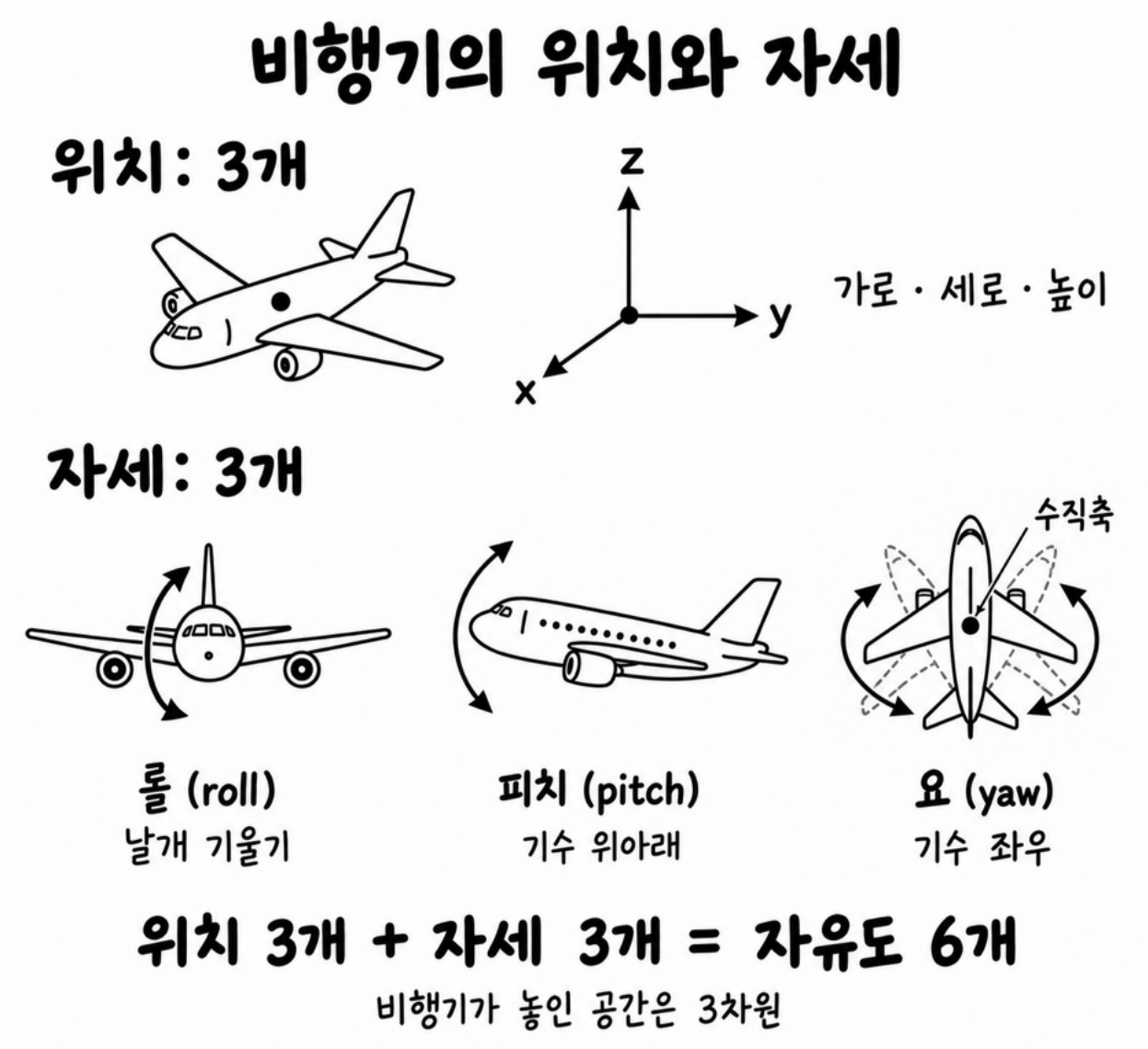


차원은 무언가를 표현할 때 필요한 정보의 개수이다. 조금 더 정확히 말하면, 서로 독립적인 정보가 몇 개 필요한지를 뜻한다. 직선 위의 한 점을 나타내려면 기준점에서 얼마나 떨어져 있는지만 알면 된다. 필요한 정보가 하나이므로 1 차원이다. 평면 위의 한 점을 나타내려면 가로 위치와 세로 위치라는 두 개의 정보가 필요하다. 그래서 평면은 2 차원이다. 여기에 높이까지 알아야 위치를 정할 수 있다면 3 차원이 된다. 이때 중요한 것은 '독립적'이라는 말이다. 같은 길이를 센티미터와 미터로 각각 적는다고 해서 차원이 늘어나지는 않는다. 하나를 알면 다른 하나는 계산할 수 있기 때문이다.



차원을 표현하는 방법은 하나만 있는 것이 아니다. 예를 들어, 2 차원 평면 위의 한 점은 **직교좌표**로 나타낼 수도 있고, **극좌표**로 나타낼 수도 있다. 직교좌표에서는 가로 위치와 세로 위치인 (x, y)로 점을 표현한다. 극좌표에서는 원점으로부터의 거리와 방향을 나타내는 각도인 (r, theta)로 같은 점을 표현한다. 표현 방법은 다르지만, 두 경우 모두 점의 위치를 정하는 데 필요한 독립적인 정보는 **두 개**다.

3 차원에서 무엇을 표현하든 세 개의 정보만 있으면 될까? 그렇지는 않다. **점의 위치**를 나타내는 데에는 세 개의 좌표가 필요하다. 하지만 위치뿐 아니라 방향까지 표현하려면 이야기가 달라진다. 작은 비행기 모형을 떠올려 보자. 비행기의 중심이 어디에 있는지는 \(x, y, z\)라는 세 개의 정보로 나타낼 수 있다. 그러나 중심이 같은 위치에 있더라도 비행기의 기수는 위나 아래를 향할 수 있고, 왼쪽이나 오른쪽을 향할 수도 있다. 날개가 한쪽으로 기울어질 수도 있다. 비행기의 자세를 정하려면 이 세 가지 회전을 나타내는 정보가 더 필요하다. 앞뒤 축을 중심으로 기울어지는 **롤(roll)**, 기수가 위아래로 움직이는 **피치(pitch)**, 기수가 좌우로 방향을 바꾸는 요(yaw)다.



따라서 크기와 모양이 고정된 비행기 모형의 위치와 자세를 나타내려면, 위치에 관한 정보 세 개와 회전에 관한 정보 세 개, 모두 **여섯 개의 독립적인 정보**가 필요하다. 이를 비행기 모형이 **6 개의 자유도**를 가진다고 표현한다. 비행기가 놓인 공간은 여전히 3 차원이다. 다만 그 공간 안에서 비행기의 위치와 자세를 함께 표현하려면 여섯 개의 정보가 필요한 것이다.

차원이 늘어난다는 것은 기존의 정보만으로는 알 수 없는 새로운 정보가 필요해진다는 뜻이다. 따라서 차원은 공간에만 쓰이는 개념이 아니다. 어떤 사람을 키와 몸무게로 표현하면 그 사람을 두 개의 좌표를 가진 점으로 나타낼 수 있다. 여기에 나이까지 더하면 세

개의 좌표를 가진 점이 된다. 이렇게 차원은 우리가 대상을 어떤 정보로 표현하는지와 연결된다. 같은 대상이라도 무엇을 담아내려 하느냐에 따라 표현에 필요한 차원은 달라질 수 있다.

5. 인공지능과 벡터

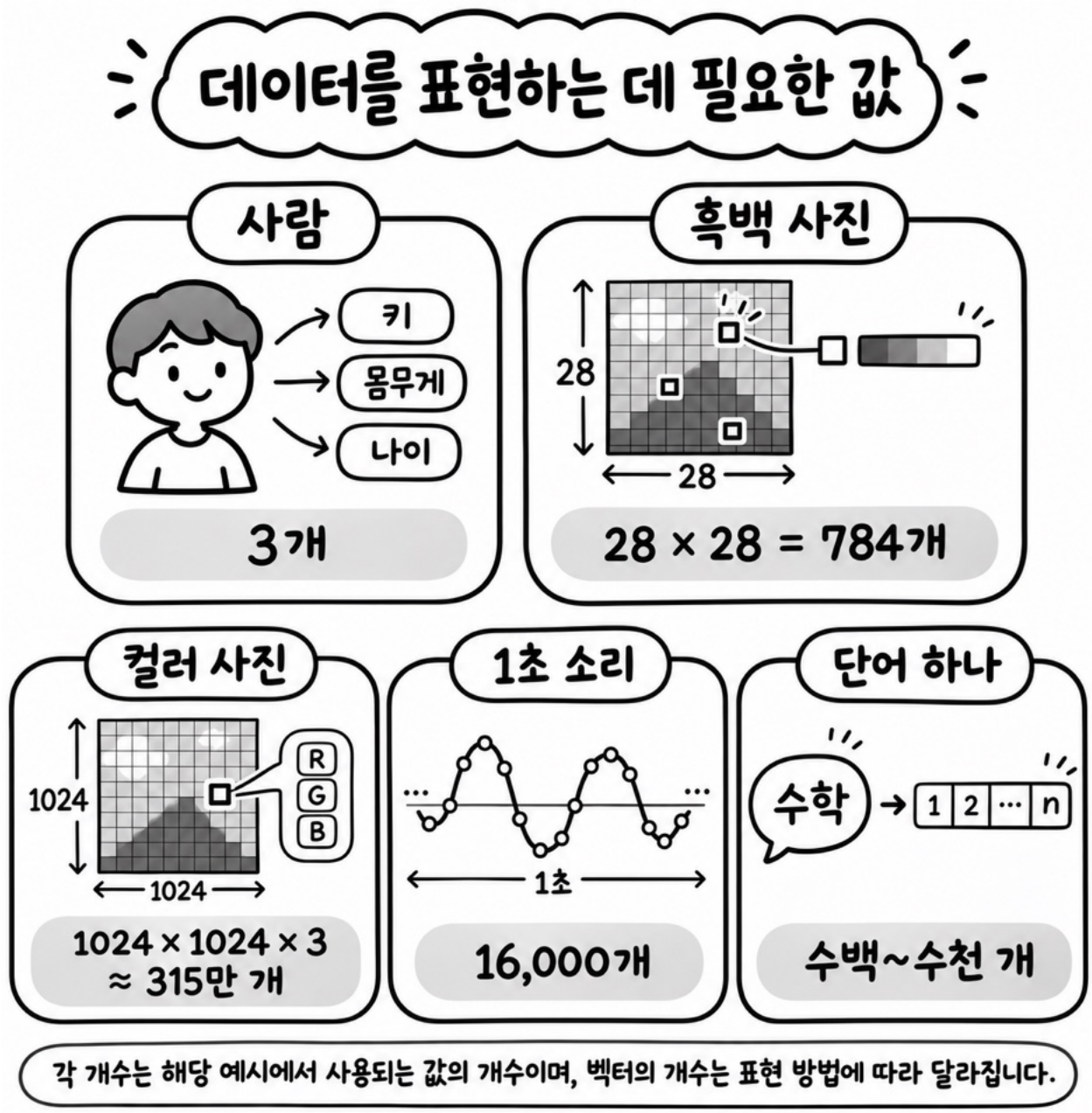
앞에서 인공지능은 입력을 받아 출력을 내놓는 거대한 함수라고 했다. 그런데 이 함수는 사진도, 문장도, 목소리도 그대로 받지 못한다. 컴퓨터 안에서 계산되는 것은 오직 숫자뿐이기 때문이다. 그래서 인공지능에 무언가를 넣으려면 먼저 그것을 숫자로 바꿔야 하고, 인공지능이 내놓는 답도 숫자의 형태로 나온다.

이때 숫자 하나로는 대개 부족하다. 사람 한 명만 해도 키, 몸무게, 나이처럼 여러 정보가 필요하다. 그래서 여러 개의 숫자를 정해진 순서대로 한 줄로 늘어놓는다. 예를 들어 키 172cm, 몸무게 65kg, 나이 30 세인 사람은 (172, 65, 30)이라는 숫자열이 된다. 이렇게 순서가 정해진 숫자의 묶음을 **벡터**라고 부른다.

벡터에 들어 있는 숫자의 개수가 그 벡터의 **차원**이다. (172, 65, 30)은 숫자가 세 개이므로 3 차원 벡터다. 앞 장에서 차원을 '표현에 필요한 독립적인 정보의 개수'라고 했는데, 벡터는 그 정보들을 실제로 적어 놓은 목록인 셈이다. 순서가 중요하다는 점도 기억해 두자. (65, 172, 30)은 몸무게가 172kg 인 전혀 다른 사람이 된다.

벡터는 공간 속의 **한 점**으로 볼 수도 있다. 2 차원 벡터 (3, 2)는 평면에서 가로로 3, 세로로 2 만큼 간 곳에 찍힌 점이고, 3 차원 벡터는 방 안의 한 점이다. 숫자가 100 개인 100 차원 벡터는 우리가 머릿속에 그릴 수는 없지만, 수학적으로는 100 개의 축을 가진 공간 속 한 점으로 똑같이 다룰 수 있다. 사람 한 명이 3 차원 공간의 점 하나가 되듯, 사진 한 장, 문장 하나도 아주 높은 차원 공간의 점 하나가 된다.

인공지능에 넣고 싶은 데이터는 모두 벡터로 바꿔야 한다. 어떤 데이터는 처음부터 숫자이고, 어떤 데이터는 숫자로 바꾸는 약속이 필요하다.

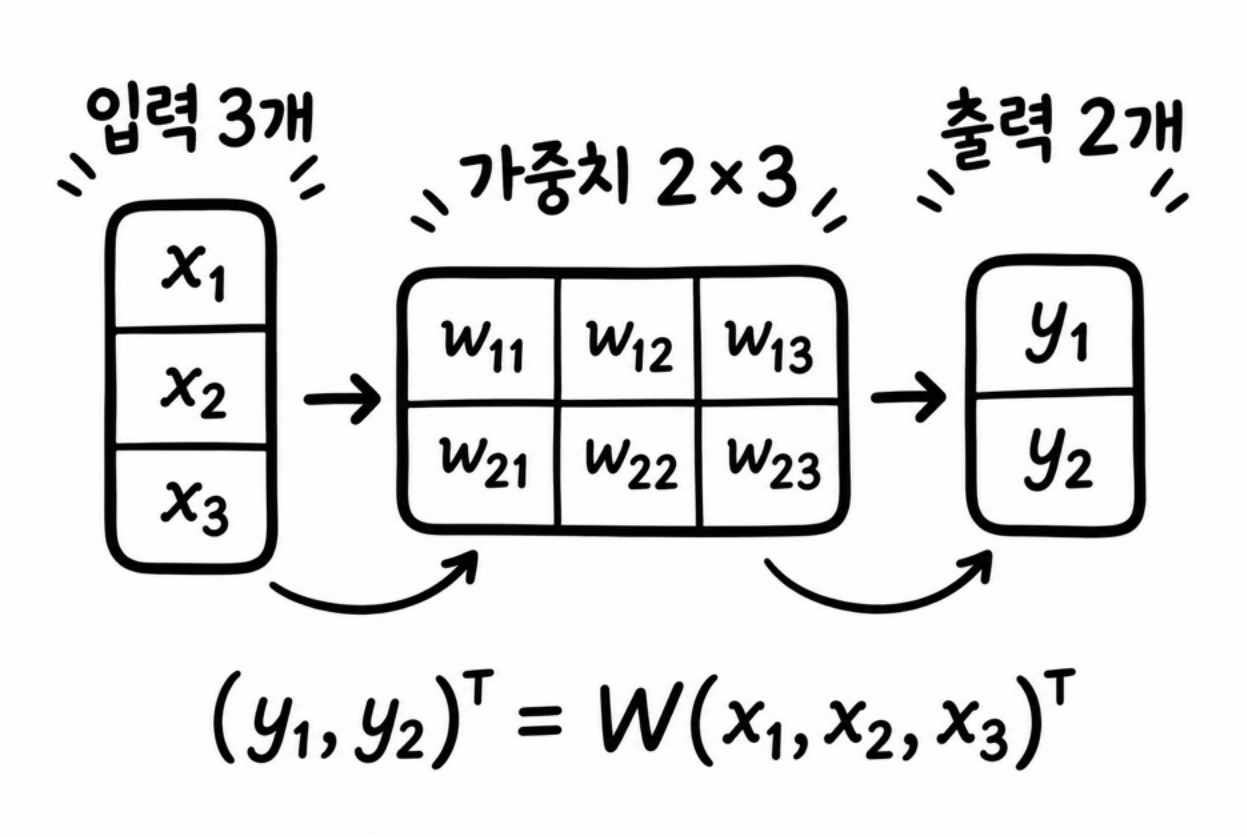


사진은 픽셀들의 모자이크이고, 픽셀마다 밝기나 색을 나타내는 숫자가 있다. 이 숫자들을 한 줄로 펼치면 사진 한 장이 벡터 하나가 된다. 소리는 공기의 떨림을 아주 짧은 간격으로 측정한 값의 나열이다.

글은 조금 다르다. '고양이'라는 단어에는 처음부터 정해진 숫자가 없다. 그래서 인공지능은 먼저 문장을 토큰이라는 조각으로 나누고, 각 토큰에 번호를 붙인 뒤, 그 번호마다 학습으로 얻은 숫자열을 연결한다. 이 숫자열을 **임베딩**이라고 부른다. 문장 하나는 결국 벡터 여러 개가 순서대로 이어진 것이 된다.

여기서 중요한 것은 같은 종류의 데이터는 같은 차원으로 바꿔야 한다는 점이다. 784 차원을 받도록 만든 인공지능에 783 개의 숫자를 넣을 수는 없다. 그래서 크기가 제각각인 사진은 먼저 같은 크기로 맞춘 뒤에 넣는다.

벡터를 받아 벡터를 내놓는 가장 기본적인 계산이 **행렬 곱셈**이다. 행렬은 숫자를 가로와 세로로 배열한 표다. 입력 벡터가 n 차원이고 출력 벡터가 m 차원이라면, 그 사이를 잇는 행렬은 m 행 n 열, 즉 m×n 개의 숫자를 가진다.

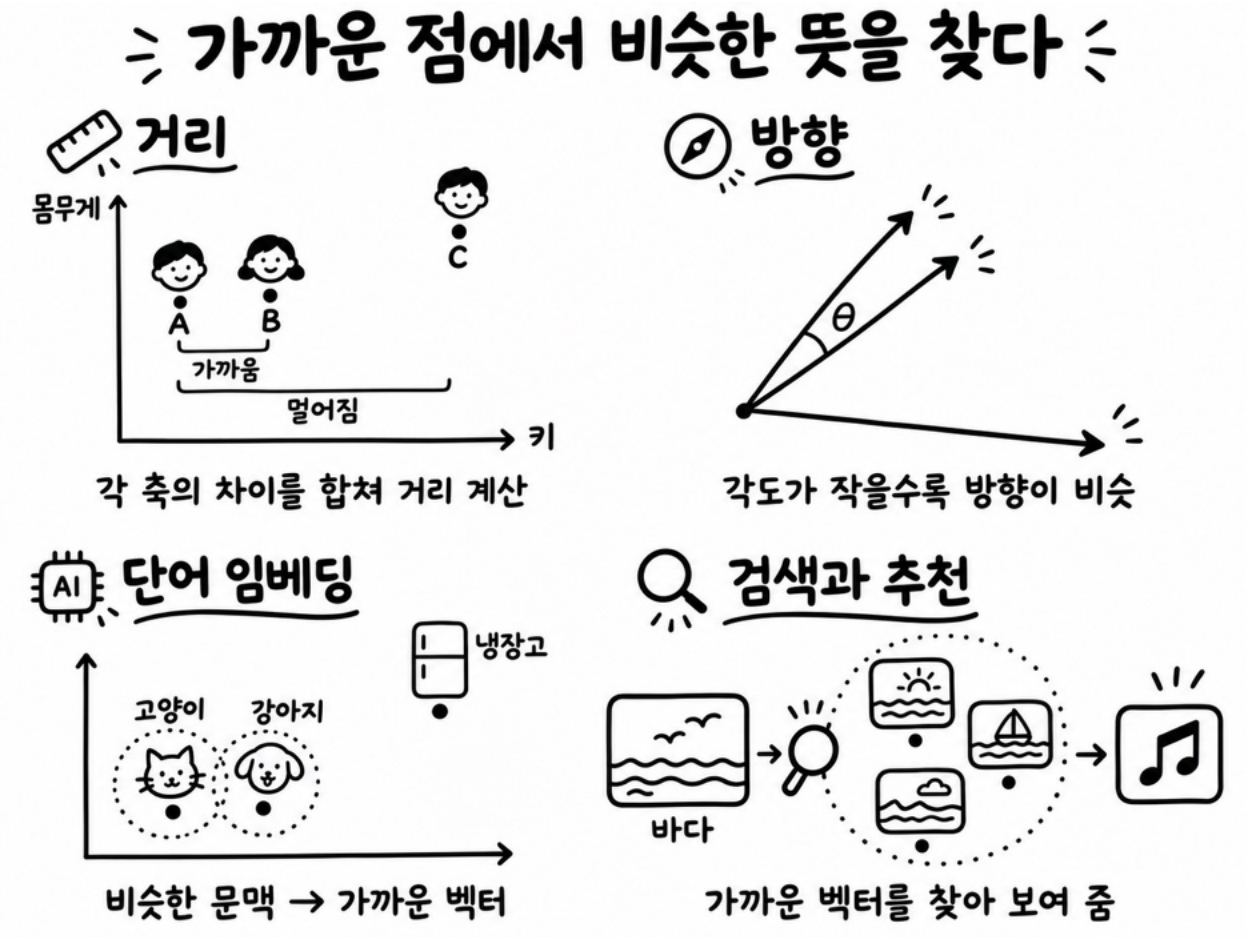


위 식은 3 차원 입력을 2 차원 출력으로 바꾸는 2×3 행렬이다. 출력의 첫 숫자 y1 은 w11×x1 + w12×x2 + w13×x3 으로 계산한다. 입력의 각 숫자에 가중치를 곱해 더하는 것이며, 가중치가 큰 입력일수록 결과에 큰 영향을 준다.

이 행렬 안의 숫자들이 바로 인공지능의 **가중치**다. 학습이란 이 숫자들을 조금씩 고쳐 원하는 출력이 나오게 만드는 과정이고, 이때 어느 방향으로 고칠지는 앞 장의 미분이 알려 준다. 784 차원 손글씨 사진을 10 개 숫자(0~9)에 대한 점수로 바꾸는 가장 단순한 인공지능도 10×784 = 7,840 개의 가중치를 가진다.

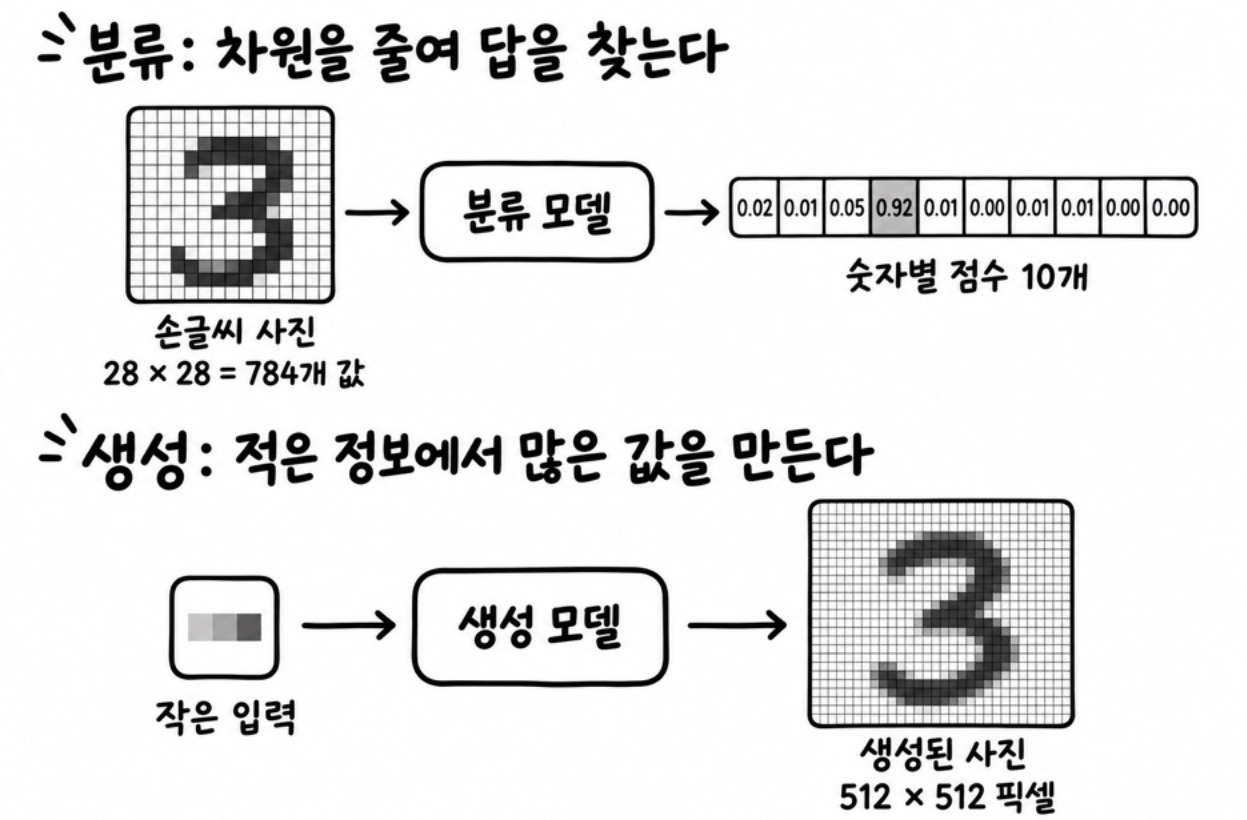
실제 인공지능은 행렬 하나가 아니라 여러 층을 격격이 쌓는다. 행렬을 곱하고, 그 결과를 살짝 구부리는 간단한 함수(비선형 함수)에 통과시킨 뒤, 다시 다음 행렬을 곱하는 일을 반복한다. 구부리는 단계가 없다면 행렬 여러 개를 곱해도 결국 행렬 하나와 같아져서, 복잡한 관계를 담을 수 없다. 그래서 엄밀하게는 "인공지능은 행렬과 비선형 함수를 번갈아 쌓은 함수"라고 해야 한다. 그래도 가중치가 담긴 곳, 계산의 대부분이 일어나는 곳은 행렬이다. 수천억 개의 가중치를 가진 거대 언어 모델도 안을 들여다보면 이런 행렬의 거대한 묶음이다.

이 구조에서 한 가지가 분명해진다. 행렬은 벡터만 곱할 수 있다. 그래서 인공지능의 입력과 출력은 반드시 벡터, 즉 정해진 길이의 숫자열이어야 한다. 첫 행렬의 열 개수가 입력의 차원을, 마지막 행렬의 행 개수가 출력의 차원을 정한다.



데이터가 공간 속의 점이 되면, 점과 점 사이의 **거리**를 잴 수 있다. 키와 몸무게가 비슷한 두 사람은 2 차원 평면에서 가까이 찍히고, 차이가 큰 사람은 멀리 찍힌다. 차원이 늘어도 계산

방법은 같다. 각 축의 차이를 제곱해 모두 더하고 제곱근을 구하면 된다. 두 벡터가 가리키는 방향이 얼마나 비슷한지를 각도로 재기도 한다.
임베딩이 힘을 발휘하는 것이 여기다. 잘 학습된 인공지능에서는 '고양이'와 '강아지'의 벡터가 가깝고, '고양이'와 '냉장고'의 벡터는 멀다. 누구도 이 자리를 정해 주지 않았다. 비슷한 문맥에서 쓰이는 단어들이 학습 과정에서 저절로 가까운 자리로 모인 것이다. 그래서 벡터 공간에서는 '뜻이 비슷하다'는 말이 '거리가 가깝다'는 계산으로 바뀐다.
이 성질은 이미 우리 일상에 들어와 있다. 사진 앱에서 '바다'를 검색하면 사진마다 달린 벡터 가운데 '바다'라는 말의 벡터에 가까운 것을 찾아 보여 준다. 음악이나 상품 추천도 내가 좋아한 것의 벡터 근처에 있는 점들을 고르는 일이다.



인공지능이 하는 일은 입력과 출력의 차원을 비교하면 크게 두 방향으로 나뉜다. 높은 차원을 낮은 차원으로 줄이는 일이 분류와 인식이고, 낮은 차원을 높은 차원으로 늘리는 일이 생성이다.

분류는 많은 숫자를 받아 적은 숫자를 내놓는다. 784 차원의 손글씨 사진을 받아, 0 부터 9 까지 각 숫자일 확률 10 개를 내놓는 식이다. 가장 큰 확률을 고르면 그것이 답이 된다. 스팸

메일 걸러내기는 출력이 단 1 차원, 즉 '스팸일 확률' 숫자 하나다. 이 과정은 미분 장에서 본 우리 뇌와 닮았다. 수백만 개의 빛 신호에서 판단에 필요한 핵심만 남기고 나머지는 버리는 압축이다.

생성은 방향이 반대다. 적은 숫자에서 출발해 많은 숫자를 만들어 낸다. 예를 들어 얼굴 사진을 만드는 생성 모델은 512 개 정도의 숫자(잠재 벡터)를 받아 수백만 개의 픽셀 값을 채워 넣는다. 잠재 벡터의 숫자를 조금 바꾸면 미소가 살짝 바뀌거나 머리카락이 길어지는 식으로 사진이 달라진다. 입력에 담기지 않은 나머지 수백만 개의 숫자는 인공지능이 학습한 '그럴듯함'으로 채워진다.

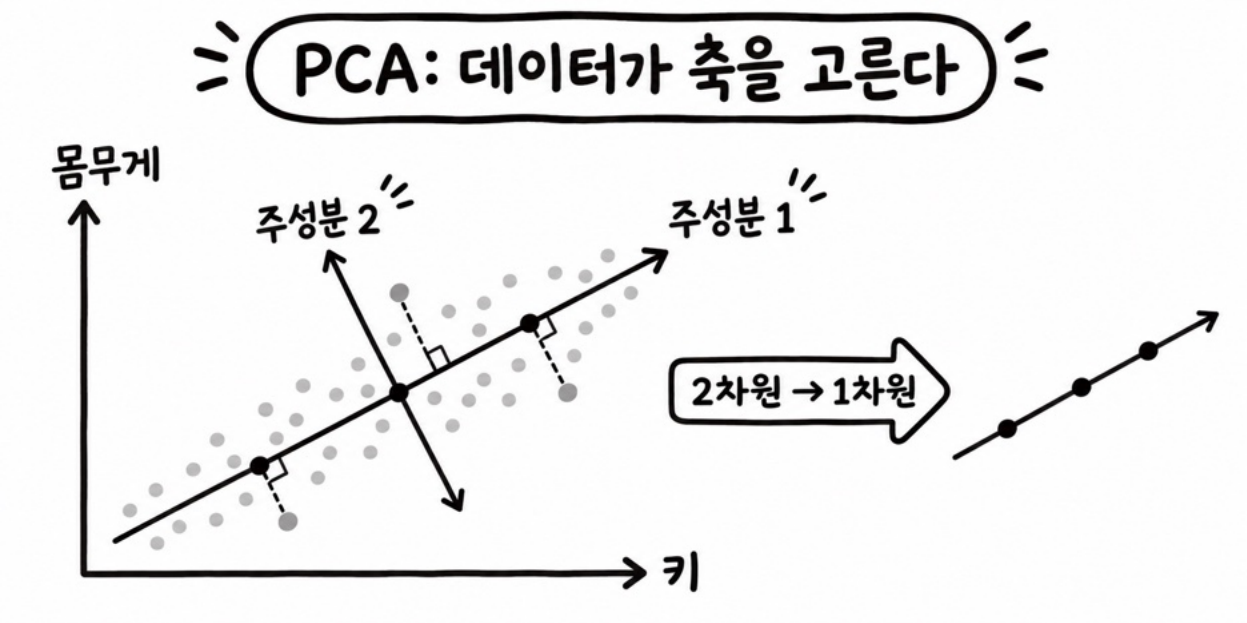
차원의 축소

데이터를 벡터로 바꾸면 차원이 금방 커진다. 컨러 사진 한 장이 약 315 만 차원이고, 소리 1 초가 16,000 차원이다. 그러나 이 숫자들이 모두 제각각 새로운 정보를 담고 있지는 않다. **차원 축소**란 본질적인 정보는 남기고 숫자의 개수를 줄이는 일이다. 차원을 줄여야 하는 이유는 다섯 가지로 정리할 수 있다.

중복이 많다. 차원 장에서 같은 길이를 센티미터와 미터로 적어도 정보는 하나라고 했다. 실제 데이터에는 이렇게 서로 따라 움직이는 숫자가 많다. 사진에서 바로 옆 픽셀끼리는 거의 같은 색이고, 키가 큰 사람은 대개 발도 크다. 독립적인 정보만 남기면 훨씬 적은 숫자로 충분하다. **계산과 저장이 비싸다.** 784 차원 입력을 100 차원으로 바꾸는 행렬에는 78,400 개의 가중치가 필요하다. 입력이 315 만 차원이면 같은 행렬에 3 억 개가 넘는 가중치가 필요하다. 차원을 줄이면 계산, 메모리, 전송량이 함께 줄어든다. **차원의 저주.** 차원이 늘면 공간이 기하급수적으로 넓어진다. 한 축을 10 칸으로 나누면 1 차원은 10 칸, 2 차원은 100 칸, 10 차원은 100 억 칸이 된다. 데이터가 만 개 있어도 10 차원에서는 대부분의 칸이 비어 있다. 또 차원이 아주 높으면 어느 점에서나 다른 점들까지의 거리가 비슷비슷해져서, '가까우면 비슷하다'는 판단이 흐려진다. 학습에 필요한 데이터의 양도 차원과 함께 급격히 늘어난다. **잡음이 섞여 있다.** 측정값에는 항상 자잘한 흔들림이 있다. 중요한 방향만 남기고 나머지를 버리면 잡음의 상당 부분도 함께 버려진다. **사람이 볼 수 있다.** 우리는 2 차원이나 3 차원까지만 그림으로 볼 수 있다. 수천 차원의 데이터를 두세 차원으로 줄여 그리면, 어떤 데이터끼리 모여 있는지 눈으로 확인할 수 있다.

차원 축소가 가능한 이유는 앞에서 본 다양체에 있다. 진짜 같은 얼굴 사진은 315 만 차원 공간 전체가 아니라 그 안의 낮은 차원 면 위에 모여 있다. 차원 축소는 그 면의 좌표를 찾는 일이다. 문제는 그 좌표를 누가, 어떻게 정하느냐이고, 여기서 세 가지 방법이 나뉜다.

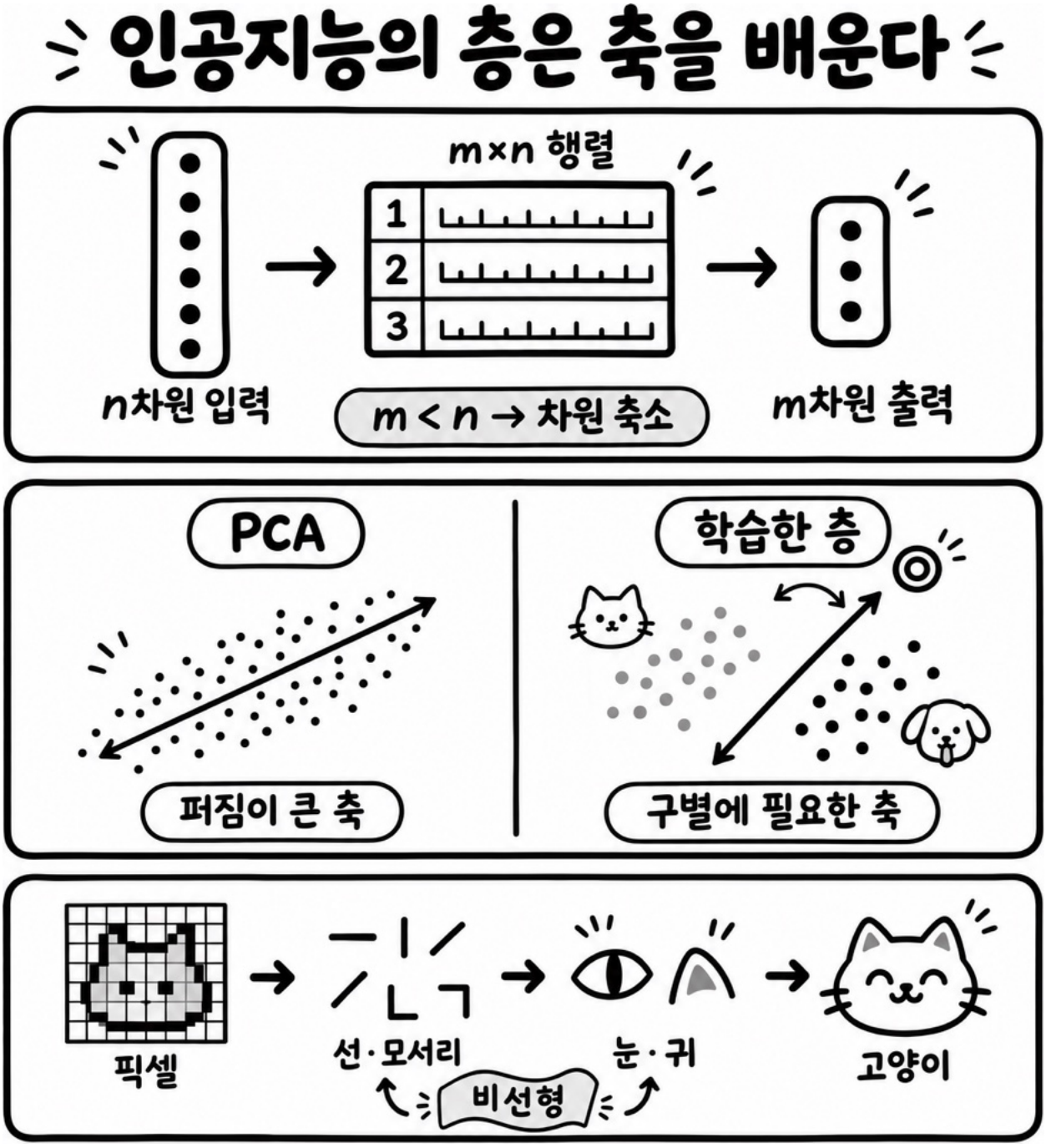
가장 오래된 방법은 그 분야를 잘 아는 사람이 '이것만 보면 된다'는 숫자를 직접 골라 계산하는 것이다. 이렇게 고른 숫자를 특징(feature)이라고 하고, 특징을 설계하는 일을 특징 공학이라고 부른다. **소리**: 푸리에 변환으로 주파수 성분을 구한 뒤, 사람의 귀가 민감한 대역을 중심으로 묶는다. 오랫동안 음성 인식에 쓰인 MFCC 는 짧은 소리 조각 수백 개의 숫자를 13 개 안팎의 계수로 줄였다. **사진**: 미분 장에서 본 것처럼 밝기의 차이(공간 미분)로 윤곽선을 뽑고, 윤곽선이 어느 방향으로 많이 나 있는지 세어 히스토그램으로 만든다. 사람 검출에 널리 쓰였던 HOG 가 이런 방식이다. **기계 센서**: 1 초에 수천 번 잰 진동 값을 그대로 쓰지 않고, 평균, 최대값, 흔들림의 크기, 특정 주파수의 세기 같은 몇 개의 숫자만 뽑는다. **압축**: JPEG 과 MP3 도 넓게 보면 같은 생각이다. 사람의 눈과 귀가 둑감한 주파수의 정보를 줄여 숫자의 개수를 줄인다. 이 방법의 장점은 **해석이 된다**는 것이다. 남은 숫자마다 '이건 평균 진동의 세기'처럼 뜻이 분명하고, 데이터가 적어도 잘 동작한다. 단점은 **사람의 지식을 넘지 못한다**는 것이다. 고양이와 강아지를 구별하는 특징을 말로 적어 보라고 하면 금방 막히듯, 사람이 설명할 수 없는 특징은 설계할 수도 없다. 문제가 바뀔 때마다 특징을 새로 설계해야 한다는 부담도 크다.



주성분 분석(PCA, Principal Component Analysis)은 사람이 특징을 고르는 대신, 데이터 스스로 가장 중요한 방향을 드러내게 하는 방법이다. 여기서 중요한 방향이란 **데이터가 가장 넓게 퍼져 있는 방향**이다. 넓게 퍼져 있다는 것은 그 방향으로 데이터끼리 차이가 크다는 뜻이고, 그 차이가 곳 정보다.

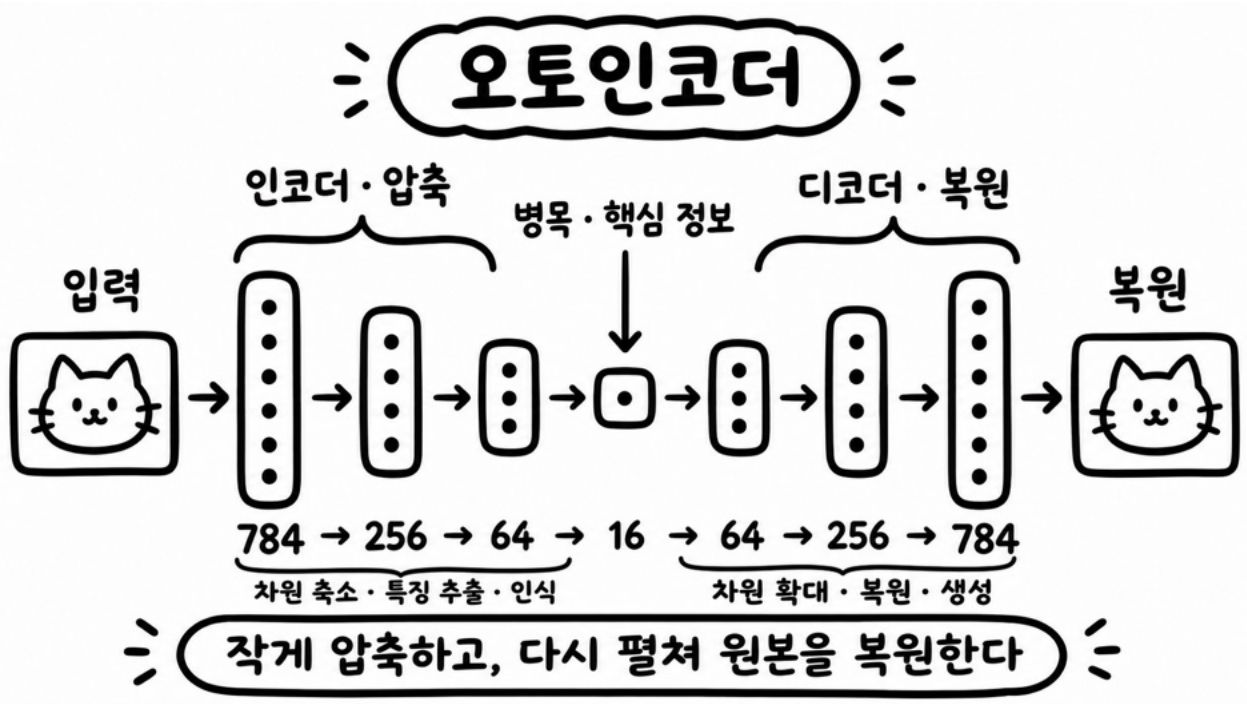
사람들의 키와 몸무게를 점으로 찍으면 오른쪽 위로 길게 늘어선 타원 모양이 된다. 키가 큰 사람은 대체로 몸무게도 나가기 때문이다. 그래서 키와 몸무게를 따로 적는 대신, 점들이 길게 늘어선 대각선 방향에 새 축을 하나 까는 편이 낫다. 이 축이 **주성분 1** 이며, '체격이 크다/작다'라는 뜻으로 읽을 수 있다. 주성분 1 에 직각인 방향이 주성분 2 이고, 같은 키에서 더 마른 편인지 통통한 편인지를 나타낸다. 점들이 이 방향으로는 조금밖에 퍼져 있지 않다면, 주성분 2 를 버리고 주성분 1 하나만 남겨도 정보를 크게 잃지 않는다. 2 차원이 1 차원으로 줄어든 것이다. 차원이 높아도 방법은 같다. 가장 넓게 퍼진 방향을 찾고, 그것과 직각인 방향 가운데 다음으로 넓게 퍼진 방향을 찾는 일을 반복한다. 각 주성분이 전체 퍼짐 가운데 얼마를 설명하는지도 계산되므로, '앞의 50 개 축으로 퍼짐의 95%를 설명한다'처럼 몇 개를 남길지 정할 수 있다. 수학적으로는 데이터가 함께 변하는 정도를 모은 행렬(공분산 행렬)의 고유벡터를 구하는 일이다. 얼굴 사진에 PCA 를 적용하면 수만 차원의 사진이 수십~수백 개의 '기본 얼굴'을 섞는 비율로 표현된다.

PCA 가 하는 일을 벡터와 행렬의 말로 하면, 좌표축을 돌린 뒤 중요하지 않은 축을 버리는 것이다. 편지 한 장을 비스듬히 놓인 책상 위에서 돌려 보며 가장 길어 보이는 방향을 찾는 것과 비슷하다. 이 모든 과정은 행렬 하나를 곱하는 것으로 끝난다. 한계도 여기서 나온다. PCA 는 직선과 평면처럼 **곧은 축**만 찾는다. 데이터가 돌돌 말린 달팽이 모양이나 구겨진 종이 위에 놓여 있다면, 곧은 축으로는 그 구조를 펼칠 수 없다. 또 '넓게 퍼진 방향'이 항상 '우리가 원하는 방향'은 아니다. 고양이와 강아지를 가르는 데 필요한 차이가 아주 작은 방향에 숨어 있을 수도 있다.



인공지능의 한 층은 행렬 곱셈이다. n 차원 입력에 m×n 행렬을 곱하면 m 차원 출력이 나오므로, m 이 n 보다 작으면 그 층은 이미 차원 축소를 하고 있다. 행렬의 각 행은 '이 방향으로 얼마나 가 있나'를 재는 축 하나다. PCA 도 행렬 하나를 곱하는 일이었다는 점을 떠올리면, 둘은 같은 모양의 계산이다. 차이는 두 가지다. 첫째, **축을 정하는 기준이 다르다.** PCA 는 '넓게 퍼진 방향'이라는 정해진 기준으로 축을 고른다. 인공지능의 층은 학습 목표에

맞춰 미분과 경사 하강법으로 축을 바꿔 간다. 고양이와 강아지를 구별하도록 학습하면 구별에 필요한 방향이 남고, 퍼진 정도가 커도 구별과 상관없는 배경의 밝기 같은 방향은 버려진다. 둘째, **휘어진 축도 만들 수 있다.** 층과 층 사이의 비선형 함수가 공간을 조금씩 구부리고 펴기 때문이다. 층을 여러 개 쌓으면 PCA 가 펼치지 못하던 돌돌 말린 다양체도 펼칠 수 있다. 사진을 다루는 인공지능에서는 층을 지날수록 픽셀에서 짧은 선과 모서리로, 다시 눈·귀 같은 부분으로, 마지막에 '고양이' 같은 개념으로 점점 추상적이고 작은 표현이 된다.

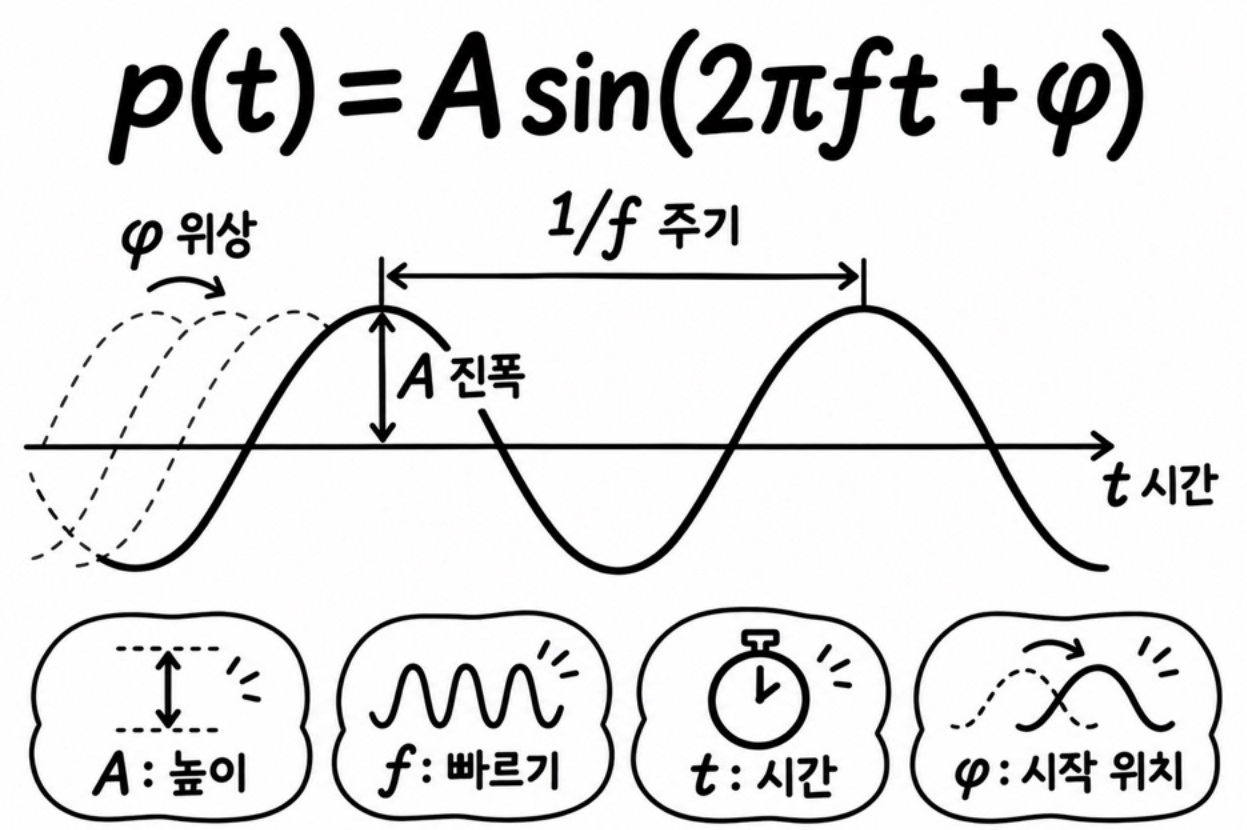


차원 축소 자체를 목표로 하는 인공지능도 있다. **오토인코더**는 가운데가 좁은 병목 모양이다. 입력을 작은 병목 벡터로 줄였다가 다시 원래 크기로 되돌리고, 출력이 입력과 같아지도록 학습한다. 병목이 좁으므로 모든 것을 통과시킬 수 없고, 복원에 꼭 필요한 핵심만 남기는 법을 스스로 배운다. 비선형 함수 없이 행렬만 쓴 오토인코더는 PCA 와 같은 공간을 찾는다는 것이 알려져 있다. 비선형을 더하면 PCA 를 휘어진 공간으로 넓힌 셈이다. 오토인코더의 앞쪽(인코더)은 분류처럼 차원을 줄이고, 뒷쪽(디코더)은 생성처럼 차원을 늘린다. 이미지 생성 인공지능 가운데에는 이렇게 압축된 공간에서 그림을 만들고 디코더로 펼치는 방식이 많다. 앞에서 본 단어 임베딩도 수만 개 단어 가운데 하나를 가리키는 거대한 번호를 수백~수천 차원으로 압축한 것이다.

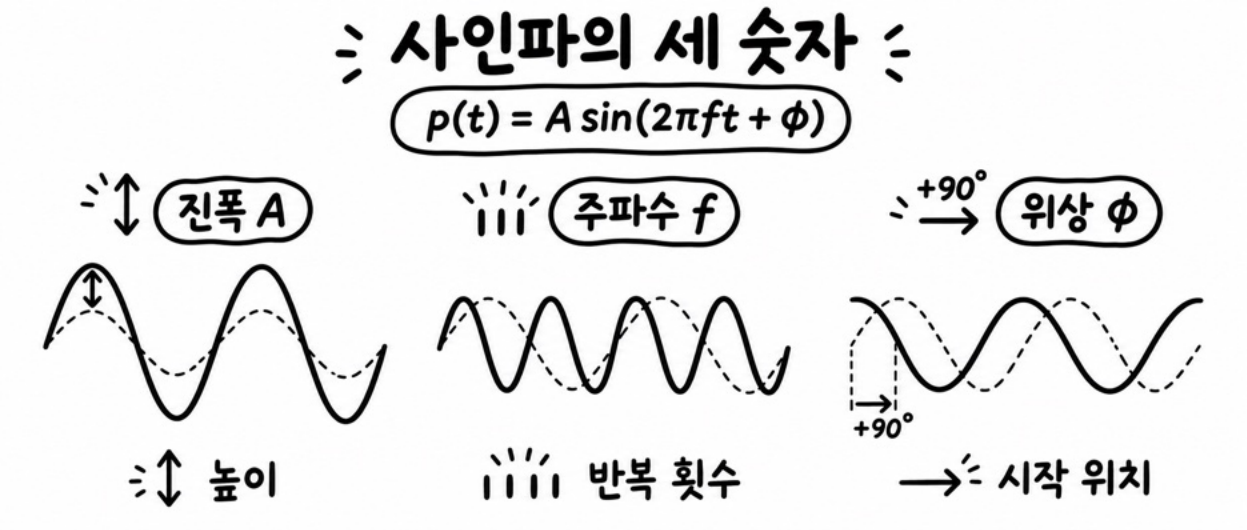
소리는 사인곡선

기타 줄을 튕기면 줄이 앞뒤로 빠르게 떨면서 주변 공기를 밀었다 당기기를 반복한다. 공기가 빽빽해진 부분과 느슨해진 부분이 번갈아 생기고, 이 압력의 변화가 공기를 타고 퍼져 나간다. 그것이 귀의 고막을 떨게 하면 우리는 소리를 듣는다. 소리는 결국 **시간에 따라 변하는 공기의 압력**이며, 시간을 입력으로 하고 압력을 출력으로 하는 함수다.
이 함수 가운데 가장 단순한 것이 **사인파**다. 원 위를 일정한 속도로 도는 점의 높이를 시간에 따라 기록하면 사인 곡선이 나온다. 소리굽쇠나 전자 기기의 '샐' 소리처럼 사인파 하나로만 이루어진 소리를 **순음**이라고 한다. 코사인파는 사인파를 옆으로 조금 밀어 놓은 것일 뿐 모양은 같다. 그래서 둘을 묶어 사인파라고 부르기도 한다.
사인파가 소리의 기본인 이유는 두 가지다. 첫째, 푸리에가 보인 것처럼 어떤 소리든 사인파의 합으로 풀 수 있다. 둘째, 우리 귀의 달팽이관 안에 있는 기저막은 위치마다 다른 빠르기의 떨림에 반응해서, 귀 자체가 소리를 사인파 성분으로 나눠 듣는다. 그래서 사인파 하나를 어떻게 기술하는지 아는 것이 소리를 이해하는 출발점이다.

모든 사인파는 같은 모양을 하고 있다. 서로 다른 점은 얼마나 크게 흔들리는가, 얼마나 빨리 반복되는가, 어디서부터 시작하는가 세 가지뿐이다. 이 세 값을 각각 **진폭**, **주파수**, **위상**이라고 부른다. 세 숫자만 알면 사인파 하나가 완전히 정해진다. 다시 말해 순음 하나는 3 차원 벡터 (진폭, 주파수, 위상)이다.



A 는 진폭, f 는 주파수, ϕ(파이)는 위상이다. t 는 초 단위의 시간이고, 2π 는 한 바퀴를 각도(라디안)로 나타낸 값이다. 2πft 는 t 초 동안 원을 몇 바퀴의 각도만큼 돌았는지를 말해 준다. 세 숫자는 들리는 방식이 다르다. 진폭을 바꾸면 소리가 커지거나 작아지고, 주파수를 바꾸면 높아지거나 낮아진다. 위상을 바꾸면 그림은 달라 보여도 귀에는 똑같이 들린다. 아래에서 하나씩 자세히 본다.



진폭(amplitude)은 파가 가운데 선에서 위아래로 얼마나 크게 흔들리는지다. 소리에서는 공기 압력이 평소보다 얼마나 높아지고 낮아지는가이며, 우리에게는 **소리의 크기**(음량)로 들린다. 진폭이 클수록 큰 소리다.

그런데 귀는 진폭을 자로 재듯 듣지 않는다. 진폭이 두 배가 되어도 두 배 크게 들리지 않고, '몇 배가 되었는가'에 반응한다. 그래서 소리의 크기는 배수를 덧셈으로 바꿔 주는 로그 눈금인 **데시벨**(dB)로 잰다. 진폭이 10 배가 되면 20dB, 2 배가 되면 약 6dB 커진다. 사람이 견딜 수 있는 가장 작은 소리에서 귀가 아플 정도의 소리까지는 약 120dB, 진폭으로는 백만 배의 차이다.

느끼는 크기는 주파수에 따라서도 달라진다. 진폭이 같아도 사람의 귀는 2,000~5,000Hz 부근을 가장 크게 듣고, 아주 낮거나 아주 높은 소리는 작게 듣는다. 진폭은 물리적인 양이고, 음량은 그것을 듣는 사람의 느낌이다.

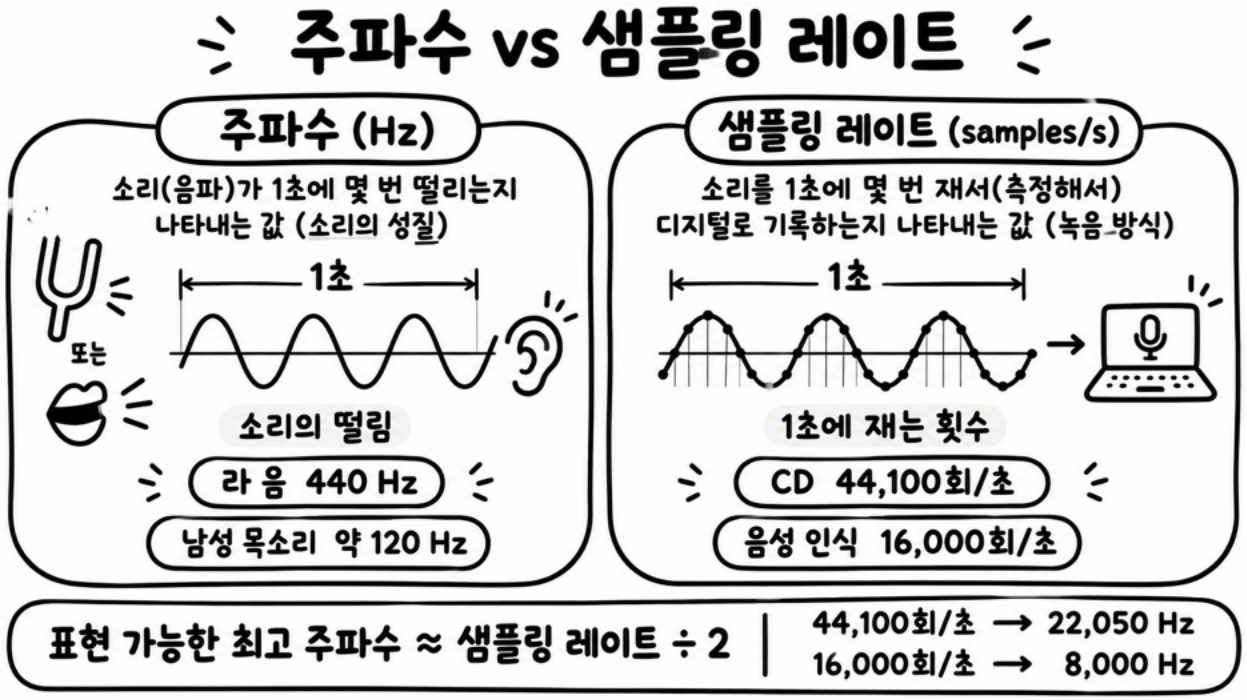
주파수(frequency)는 파가 **1 초에 몇 번 반복되는가**다. 단위는 **헤르츠**(Hz)이며, 1Hz 는 1 초에 한 번 반복된다는 뜻이다. 한 번 반복되는 데 걸리는 시간을 **주기**라고 하고, 주기는 주파수의 역수다. 440Hz 인 소리의 주기는 1/440 초, 약 2.3 밀리초다.

주파수는 우리에게 **소리의 높이**(음정)로 들린다. 빨리 떨수록 높은 소리, 천천히 떨수록 낮은 소리다.

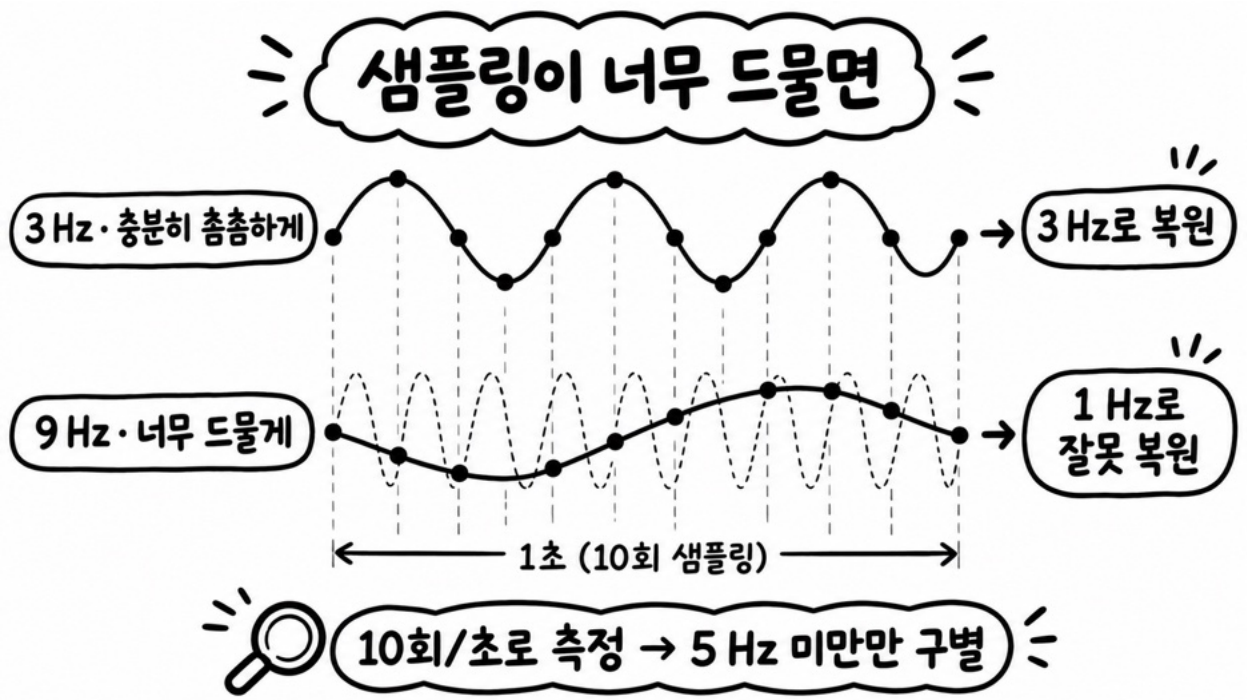
- **가청 범위**: 사람은 대략 20Hz 에서 20,000Hz 까지 듣는다. 나이가 들수록 위쪽 한계가 내려가서, 성인은 15,000Hz 위를 잘 못 듣는 경우가 많다.

- **기준음**: 오케스트라가 조율하는 '라'(A4)는 440Hz 다.
- **옥타브**: 주파수가 두 배가 되면 한 옥타브 높은 같은 이름의 음이 된다. 880Hz 는 한 옥타브 위의 '라'다. 피아노의 반음은 주파수를 약 1.06 배(2 의 12 제곱근)씩 올리므로, 12 번 올라가면 정확히 2 배가 된다. 음량처럼 음높이도 차이가 아니라 비율로 듣는 것이다.
- **목소리**: 말할 때 성대가 떨리는 기본 주파수는 성인 남성이 대체로 85~180Hz, 성인 여성이 165~255Hz 안팎이다.
- **파장**: 소리는 공기 중에서 1 초에 약 343m 를 간다. 한 주기 동안 나아가는 거리인 파장은 속도를 주파수로 나눈 값이다. 440Hz 의 파장은 약 78cm, 20Hz 는 약 17m, 20,000Hz 는 약 1.7cm 다. 낮은 소리가 벽을 돌아 잘 넘어오는 것은 파장이 길기 때문이다.

녹음 파일을 보면 '44,100Hz'나 '16kHz' 같은 숫자가 붙어 있다. 단위는 같은 Hz 지만 이것은 소리의 높이가 아니다. **샘플링 레이트**는 컴퓨터가 소리를 숫자로 바꿀 때 **1 초에 몇 번 재는가**다. 두 값 모두 '1 초에 몇 번'이라서 단위가 같을 뿐이다.



녹음은 연속된 파형 위에 일정한 간격으로 점을 찍는 일이다. 샘플링 레이트가 16,000Hz 라면 1 초에 점을 16,000 개 찍고, 그래서 1 초 소리가 16,000 차원 벡터가 된다.



점을 너무 드물게 찍으면 문제가 생긴다. 빨리 떨리는 파의 사이사이를 놓치는 바람에, 찍힌 점들이 느린 파처럼 보이게 된다. 이를 **에일리어싱**이라고 한다. 영화에서 빠르게 도는 자동차 바퀴가 느리게 도는 것처럼, 혹은 거꾸로 도는 것처럼 보이는 것도 같은 현상이다. 영화는 1 초에 24 장만 찍기 때문이다.

얼마나 촘촘히 찍어야 하는지는 **나이퀴스트 정리**가 알려 준다. 샘플링 레이트의 **절반보다 낮은 주파수**만 제대로 담을 수 있다. 한 주기에 적어도 점이 두 개(위한 번, 아래 한 번)는 찍혀야 떨림을 알아볼 수 있기 때문이다.

- CD 의 44,100Hz 는 사람이 듣는 최고 주파수인 약 20,000Hz 를 담기 위해 그 두 배보다 조금 넓게 정한 값이다. 22,050Hz 까지 담는다.
- 음성 인식과 음성 합성에 흔히 쓰는 16,000Hz 는 8,000Hz 까지 담는다. 말을 알아듣는 데 필요한 정보는 대부분 이 안에 있다.
- 전통적인 전화는 8,000Hz 로 재어 4,000Hz 아래만 전달한다. 전화 목소리가 조금 답답하게 들리고 '스'와 '프'가 헷갈리는 이유다.

그래서 녹음 장비는 샘플링 레이트의 절반보다 높은 소리를 미리 걸러 낸 뒤에 잰다. 걸러 내지 않으면 없던 낮은 소리가 녹음에 들어간다.

위상(phase)은 파가 한 주기 안에서 어느 지점에서 시작하는지다. 원 위를 도는 점으로 생각하면, 출발 지점의 각도다. 그래서 각도로 재며, 한 바퀴가 360 도(2π 라디안)다. 위상이 90 도 다르면 파가 4 분의 1 주기만큼 옆으로 밀린 것이고, 사인과 코사인이 바로 이 관계다. 180 도 다르면 위아래가 뒤집힌다.

중요한 사실은 **사람은 위상을 거의 듣지 못한다**는 것이다. 순음 하나는 언제 시작했든 똑같이 들린다. 여러 배음이 섞인 소리에서 각 배음의 위상만 바꾸면 파형의 모양은 크게 달라지지만, 음색은 거의 같게 들린다. 19 세기에 옴과 헬름홀츠가 정리한 이 관찰은, 귀가 소리를 주파수 성분으로 나누고 각 성분의 크기만 주로 전달한다는 뜻이다. 그래서 음성 인식 인공지능은 푸리에 변환 후 각 주파수의 크기만 남기고 위상은 버린 스펙트로그램을 주로 쓴다. 입력의 차원이 절반으로 줄어도 들리는 내용은 거의 잃지 않는 셈이다.

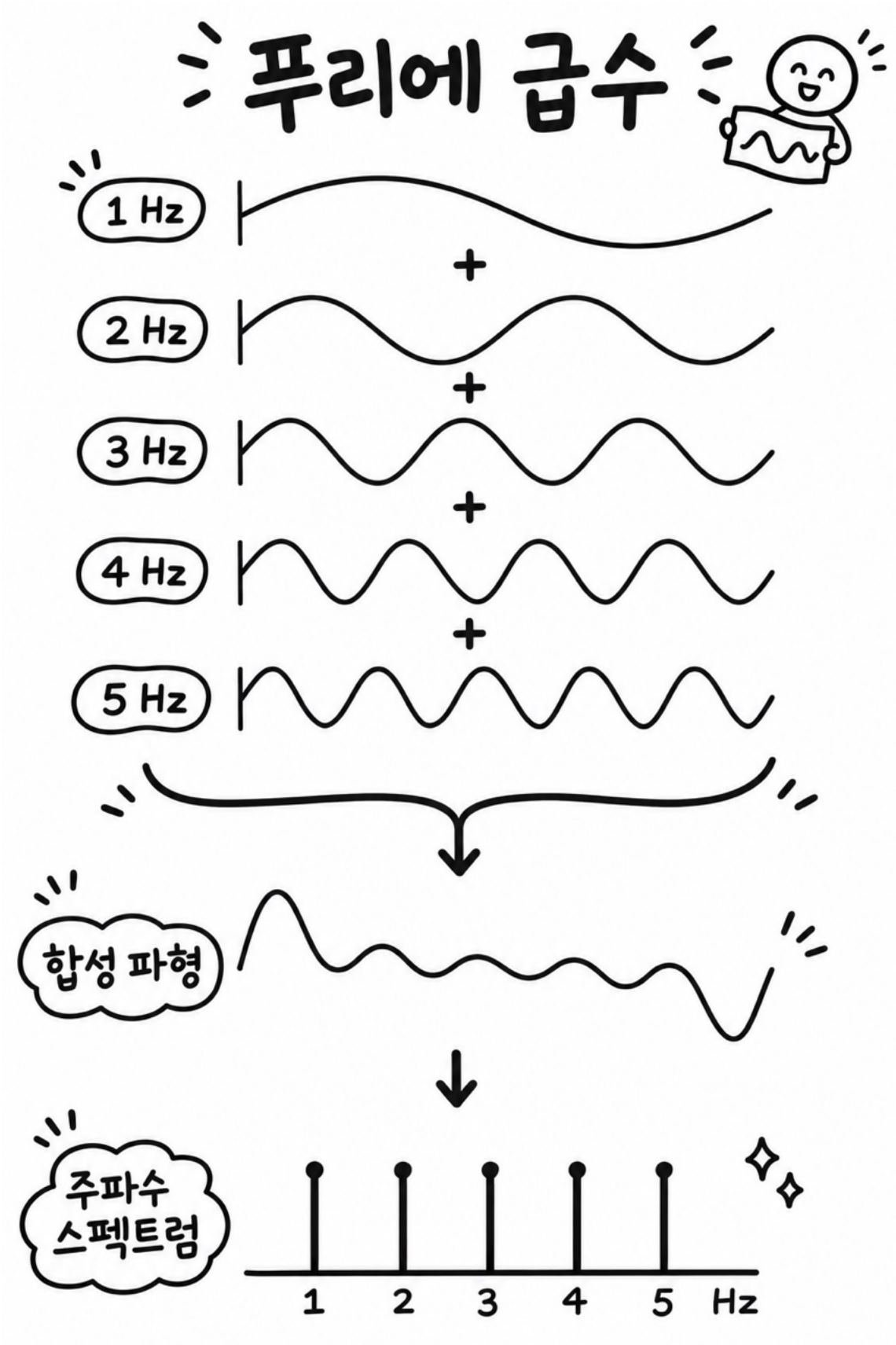
다만 위상이 아무 역할도 하지 않는 것은 아니다.

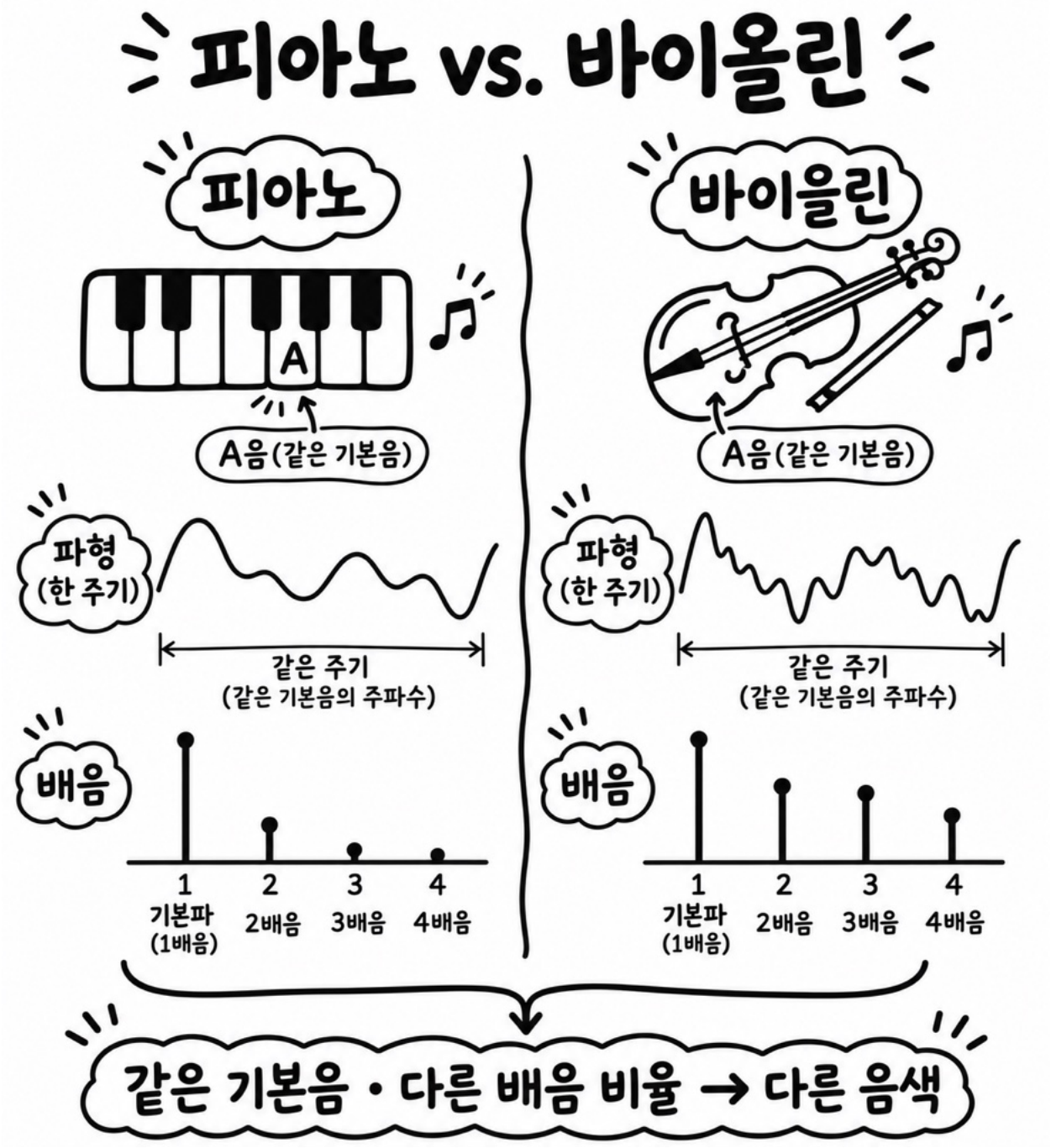
- **방향 듣기**: 오른쪽에서 오는 소리는 왼쪽 귀에 조금 늦게 도착한다. 뇌는 두 귀 사이의 이 작은 시간·위상 차이로 소리의 방향을 안다. 파장이 긴 낮은 소리(대략 1,500Hz 아래)에서 특히 중요하다. 즉 한 귀 안의 위상은 못 듣지만, 두 귀 사이의 위상 차이는 듣는다.
- **상쇄와 보강**: 같은 주파수의 두 파를 더할 때는 위상이 결정적이다. 위상이 같으면 두 배로 커지고, 180 도 다르면 서로 지워진다. 이어폰의 노이즈 캘슬링은 마이크로 들은 소음을 뒤집힌 위상으로 내보내 상쇄시킨다. 스피커 두 개의 선을 하나 거꾸로 꼽으면 낮은 소리가 빈약해지는 것도 같은 이유다.
- **소리 만들기**: 음성 합성 인공지능은 스펙트로그램을 먼저 만든 뒤 다시 소리로 바꾼다. 스펙트로그램에는 위상이 없으므로, 자연스러운 위상을 채워 넣는 단계(보코더)가 따로 필요하다. 하나의 소리에서는 위상을 못 듣지만, 시간을 따라 이어지는 조각들의 위상이 어긋나면 기계음처럼 지글거리는 소리가 난다.

순음은 실험실 밖에서 거의 들을 수 없다. 악기와 목소리는 기본 주파수의 사인파 위에 그 정수배인 배음들이 겹쳐 있다. 이때 우리가 듣는 음높이는 대개 기본 주파수이고, 배음들의 진폭 비율이 **음색**을 정한다. 같은 '라'(440Hz)라도 피아노와 바이올린이 다르게 들리는 것은 880Hz, 1,320Hz … 의 진폭이 다르기 때문이다. 반면 바람 소리나 '쉬' 같은 잡음은 특정한 배음 구조 없이 넓은 주파수가 고루 섞여 있다.

말소리도 같다. 성대의 떨림이 기본 주파수와 배음을 만들고, 입과 혀의 모양이 어떤 주파수 대역을 키우고 줄이느냐에 따라 ‘아’와 ‘이’가 갈린다. 그래서 음성 인식은 결국 시간에 따라 변하는 각 주파수의 진폭을 읽는 일이다.

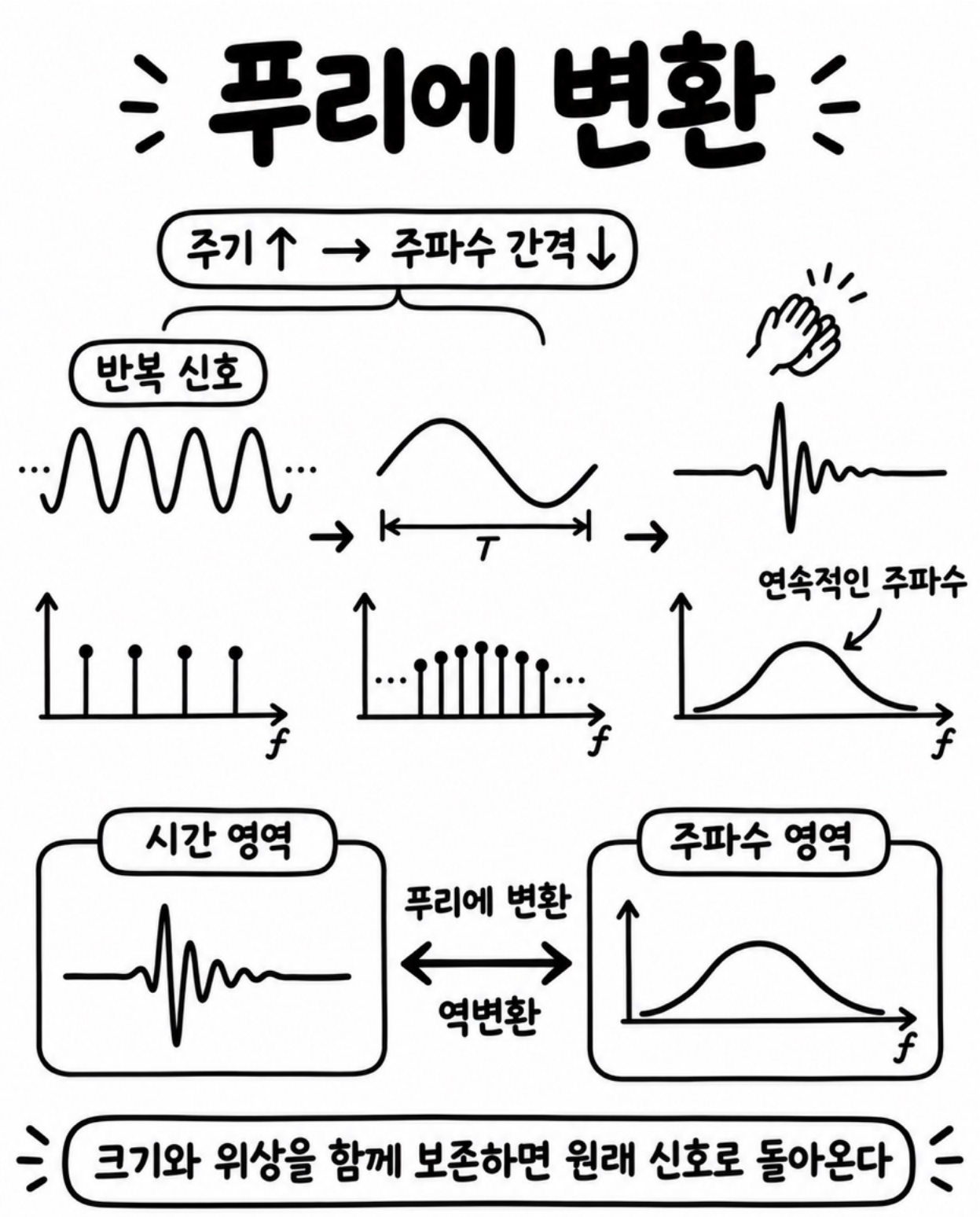
푸리에 급수 / 변환


푸리에 급수
1 Hz
2 Hz
3 Hz
4 Hz
5 Hz
합성 파형
주파수
스펙트럼
1
2
3
4
5
Hz



일정한 주기로 반복되는 함수는 어떤 모양이든 가장 단순한 물결인 사인파를 여러 개 더해서 만들 수 있다는 것이다. 이렇게 함수를 사인파의 합으로 풀어 쓴 것을 푸리에 급수라고 한다.더해지는 사인파는 아무거나가 아니다. 원래 함수와 같은 빠르기로 반복하는 파(기본파), 그보다 두 배 빠른 파, 세 배 빠른 파처럼 정수배로 빨라지는 파들만

쓴다. 음악에서는 이를 배음이라고 부른다. 피아노와 바이올린의 음색이 다른 것은 배음을 섞는 비율이 다르기 때문이다.



푸리에 급수는 반복되는 함수에만 쓸 수 있다. 그런데 사람의 말소리, 박수 소리, 지진파처럼 세상의 신호는 대부분 반복되지 않는다. 푸리에 변환은 이런 신호에도 같은 생각을 적용한다. 방법은 이렇다. 반복되지 않는 신호를, 주기가 한없이 길어서 아직 한 번도 반복되지 않은 함수라고 생각한다. 주기가 길어지면 기본파가 느려지고 배음들 사이의 간격이 좁아진다. 주기가 무한대가 되면 간격이 사라져, 정수배의 배음만이 아니라 **모든 빠르기의 사인파**가 조금씩 섞인 것이 된다. 미분에서 구간을 한없이 좁혀 평균변화율을 순간변화율로 바꾼 것과 비슷한 발상이다. 이렇게 같은 신호에 두 가지 표현이 생긴다. 시간마다 값을 적은 것을 **시간 영역**, 빠르기(주파수)마다 양을 적은 것을 **주파수 영역**이라고 한다. 주파수 영역에서 본 모습을 **스펙트럼**이라고 부른다. 스펙트럼에서 다시 시간 영역으로 돌아오는 역변환도 있고, 오가는 동안 정보는 하나도 사라지지 않는다.